

ENTREGABLE 1: DOCUMENTACIÓN INICIAL DEL PROYECTO DE SOFTWARE – SPRINT 0 Y ANÁLISIS TÉCNICO

APLICACIÓN DE INGENIERÍA DE SOFTWARE EN EL
DESARROLLO DE UN SISTEMA ERP MODULAR DE
GESTION FINANCIERA Y CONTABLE BAJO LA
METODOLOGÍA SCRUM

Curso: Taller de Procesos ERP / ERP Process Workshop

Ciclo académico: VI

Integrantes del equipo Sprint:

1. Alvis Danival Quispe Huayta – Sprint / Analista de Requisitos
2. Elias Ronald Ticona Callata – Backend Developer
3. Luis Fernando Cutiri Vilca – Frontend Developer
4. Edison David Supo Cruz – Arquitecto y Admin. Base de Datos

Fecha: 2026-1

M.Sc. Juan Carlos Pinto Larico
Docente

Resumen

El presente documento constituye el Entregable 1 del proyecto de desarrollo del Módulo de Gestión Financiera y Contable del sistema ERP modular, desarrollado en el marco del curso Taller de Procesos ERP de la Escuela Profesional de Ingeniería de Software y Sistemas de la Universidad Nacional de Juliaca (UNAJ). El proyecto aplica la metodología ágil Scrum para gestionar el ciclo de vida del software, desde la identificación de requisitos hasta la documentación técnica del Sprint 0.

El módulo abarca dos subsistemas fundamentales: el Sistema de Planillas, diseñado para el cálculo automático de sueldos, descuentos previsionales (AFP al 10 % y ONP al 13 %), Compensación por Tiempo de Servicios (CTS), gratificaciones legales (julio y diciembre) y la generación de boletas de pago en formato PDF; y el Sistema Contable SUNAT, orientado al registro de comprobantes de compra y venta, la generación automatizada del Libro de Compras y Libro de Ventas electrónicos conforme a la normativa tributaria peruana (IGV al 18 %), y la elaboración de reportes financieros como el balance general y el estado de resultados.

El equipo de desarrollo está conformado por cuatro integrantes con roles claramente definidos: Alvis Danival Quispe Huayta como Sprint/Analista de Requisitos, Elias Ronald Ticona Callata como Backend Developer, Luis Fernando Cutiri Vilca como Frontend Developer y Edison David Supo Cruz como Arquitecto y Administrador de Base de Datos. La plataforma tecnológica seleccionada emplea C# con .NET Core para la lógica de negocio, Windows Forms (WinForms) para la interfaz de escritorio y PostgreSQL 16 como motor de base de datos relacional.

Este documento presenta la planificación del Sprint 0 con la identificación detallada de requisitos funcionales y no funcionales, los diagramas UML por cada rol del equipo, el backlog inicial del producto, la especificación del modelo del sistema con sus variables y fórmulas, el proceso de normalización de base de datos hasta la forma Boyce-Codd (BCNF) y los anexos técnicos requeridos para el inicio formal del desarrollo del módulo financiero.

Contenido

Resumen	1
Introducción.....	6
Métodos de investigación y levantamiento de información.....	6
Objetivo.....	7
Objetivo general	7
Objetivos específicos	7
Visión del Producto.....	8
Módulo de Planillas	8
Módulo Contable SUNAT	9
Dashboard de Reportes Financieros	9
Autenticación y Control de Acceso.....	10
Modelo de Sistema	10
Departamento de Recursos Humanos	11
Departamento de Contabilidad.....	11
Gerencia	12
Departamento de TI/Sistemas	12
Variables del Sistema	13
Variables del Módulo de Planillas	13
Salario Bruto	13
AFP (Administradora de Fondos de Pensiones)	13
ONP (Oficina de Normalización Previsional).....	13
CTS (Compensación por Tiempo de Servicios)	14
Gratificación.....	14
Descuentos Totales.....	14
Salario Neto.....	14
Variables del Módulo Contable	15
IGV (Impuesto General a las Ventas)	15
Indicadores de Desempeño	15
Planificación – Sprint 0.....	15
Roles del equipo Scrum.....	15
Backlog inicial del producto.....	16
Historia 1: Autenticación y acceso al sistema.....	16
Historia 2: Registro de empleados.....	17

Historia 3: Cálculo de planilla mensual.....	18
Historia 4: Registro de comprobante contable.....	19
Historia 5: Generación de Libro de Compras y Ventas.....	19
Historia 6: Visualización de reportes financieros.....	20
Análisis de Requisitos.....	21
Requisitos por Rol.....	21
Alvis Danival Quispe Huayta – Sprint / Analista de Requisitos.....	21
Luis Fernando Cutiri Vilca – Frontend Developer.....	23
Elias Ronald Ticona Callata – Backend Developer.....	26
Edison David Supo Cruz – Arquitecto y Admin. Base de Datos.....	28
Matriz consolidada de Requisitos Funcionales.....	31
Matriz consolidada de Requisitos No Funcionales.....	31
Diagramas UML.....	32
Alvis Danival Quispe Huayta – Sprint / Analista de Requisitos.....	32
Diagrama de Casos de Uso – Proceso de Análisis de Requisitos.....	32
Diagrama de Secuencia – Proceso de Levantamiento de Requisitos....	35
Diagrama de Objetos – Instancias del Sprint 0.....	36
Diagrama de Actividad – Proceso Sprint 0.....	38
Elias Ronald Ticona Callata – Backend Developer.....	40
Diagrama de Casos de Uso – Capa Backend y Lógica de Negocio.....	40
Diagrama de Clases – Arquitectura Backend ASP.NET Core.....	42
Diagrama de Secuencia – Autenticación JWT y Cálculo de Planilla.....	44
Diagrama de Objetos – Instancias del Backend en Ejecución.....	46
Diagrama de Componentes – Arquitectura Backend ERP.....	47
Luis Fernando Cutiri Vilca – Frontend Developer.....	50
Diagrama de Casos de Uso – Interfaz de Usuario (WinForms).....	50
Diagrama de Clases – Patrón MVVM en WinForms.....	52
Diagrama de Secuencia – Flujo de Pantallas: Cálculo de Planilla.....	54
Diagrama de Objetos – Instancias WinForms en Tiempo de Ejecución	55
Diagrama de Estados – Navegación de Pantallas WinForms.....	57
Edison David Supo Cruz – Arquitecto y Admin. Base de Datos.....	59
Diagrama de Casos de Uso – Base de Datos PostgreSQL.....	59
Diagrama de Clases – Esquema de Base de Datos (3NF).....	61
Diagrama de Secuencia – Migración y Ejecución de Stored Procedures	64
Diagrama de Objetos – Instancias de la Base de Datos.....	66

Normalización de Base de Datos (1FN → 2FN → 3FN → BCNF).....	67
Normalización de Base de Datos (Edison David Supo Cruz).....	67
Tabla sin normalizar (No Normalizada).....	68
Primera Forma Normal (1FN).....	68
Segunda Forma Normal (2FN).....	69
Tercera Forma Normal (3FN).....	70
Esquema Final BCNF (Boyce-Codd Normal Form).....	72
Referencias.....	74
Entregables del Sprint 0.....	75
Anexo 1: Hoja de Identificación del Proyecto.....	75
Información General del Proyecto.....	75
Matriz de Requisitos Funcionales.....	75
Matriz de Requisitos No Funcionales.....	76
Cronograma Tentativo.....	76
Anexo 2: Hoja de Diseño Técnico.....	77
Introducción.....	77
Arquitectura del Sistema (Tres Capas + Patrón MVC).....	77
Componentes Principales.....	78
Diagrama de Interacción (Secuencia) - Generación del Libro de Ventas Mensual.....	79
Consideraciones Técnicas.....	79
Criterios de Aceptación Técnicos.....	80
Anexo 3: Identificación de Requisitos Funcionales.....	80
Contexto del sistema.....	80
Pasos de trabajo.....	81
Tabla de Registro de Requisitos Funcionales.....	81
Anexo 4: Definición de Requisitos No Funcionales.....	83
Contexto del sistema.....	83
Categorías de requisitos no funcionales.....	83
Pasos de trabajo.....	83
Tabla de Registro de Requisitos No Funcionales.....	84
Anexo 5: Wireframes y Pantallas del Sistema.....	85
Pantalla de Login / Inicio de Sesión.....	85
Pantalla Dashboard Principal.....	86
Pantalla Módulo Planillas - Lista de Empleados.....	87
Pantalla Formulario de Empleado.....	88

Pantalla Cálculo de Planilla.....	89
Pantalla Módulo Contable - Lista de Comprobantes.....	90
Pantalla Generación de Libros SUNAT	91
Pantalla Dashboard de Reportes Financieros	91
Pantalla de Administración de Usuarios	92

Introducción

El presente proyecto corresponde al desarrollo del Módulo de Gestión Financiera y Contable del Sistema ERP modular, en el marco del curso Taller de Procesos ERP de la Escuela Profesional de Ingeniería de Software y Sistemas de la Universidad Nacional de Juliaca (UNAJ). El objetivo central es aplicar los fundamentos de la ingeniería de software y la metodología ágil Scrum para construir un sistema de escritorio que automatice los procesos de planillas salariales y la gestión contable bajo la normativa tributaria peruana (SUNAT).

Métodos de investigación y levantamiento de información

Para comprender a profundidad los procesos de negocio del módulo financiero-contable, se emplearon los siguientes métodos de recolección y análisis de información:

- Entrevistas a stakeholders: Se realizaron entrevistas semiestructuradas con actores clave del dominio financiero-contable, incluyendo al jefe del departamento de Recursos Humanos, al contador general de una empresa mediana y al gerente financiero. Cada entrevista se documentó en actas que registran preguntas, respuestas, observaciones y requisitos emergentes. El análisis de estas entrevistas permitió identificar los flujos de trabajo reales, las fuentes de datos, los puntos de dolor en los procesos manuales actuales y las expectativas de automatización.
- Análisis de la bitácora del proyecto: Se revisó exhaustivamente la bitácora del curso Taller de Procesos ERP, que contiene el registro cronológico de las actividades del equipo, las decisiones de diseño, los acuerdos de las reuniones de planificación y las actas de las daily Scrum meetings. Este análisis permitió reconstruir la línea base del proyecto, identificar los riesgos tempranos y documentar la evolución de los requisitos desde la visión inicial hasta la especificación final del Sprint 0.
- Revisión de la normativa SUNAT: Se estudiaron las disposiciones legales vigentes emitidas por la Superintendencia Nacional de Aduanas y de Administración Tributaria (SUNAT) respecto a los Libros Electrónicos (Resolución de Superintendencia N.° 286-2009/SUNAT y modificatorias), las tasas del Impuesto General a las Ventas (IGV al 18 %), el Impuesto a la Renta de quinta categoría, y los regímenes previsionales AFP (10 % de la remuneración asegurable) y ONP (13 % de la remuneración). Esta revisión garantiza que el

sistema cumpla con la legislación tributaria peruana vigente al momento del desarrollo.

- Reuniones de equipo con metodología Scrum: El equipo sostuvo reuniones diarias de sincronización (Daily Scrum) de 15 minutos, una reunión de planificación del Sprint 0 (Sprint Planning) de 4 horas y una reunión de revisión del backlog (Backlog Refinement) de 2 horas. En estas reuniones se aplicaron los principios de Scrum descritos por Schwaber y Sutherland (2020): transparencia, inspección y adaptación. Cada reunión generó artefactos tangibles: Sprint Backlog, Definition of Done y criterios de aceptación para cada historia de usuario.
- Análisis documental de sistemas ERP existentes: Se revisaron las especificaciones funcionales de sistemas ERP comerciales (SAP Financials, Oracle E-Business Suite Financials, Odoo Accounting) y de código abierto (Tryton, ERPNext) para identificar las mejores prácticas en la implementación de módulos financieros y contables. Este análisis comparativo informó las decisiones de diseño arquitectónico y la granularidad de los requisitos funcionales.

Objetivo

Objetivo general

Aplicar los procesos, técnicas y herramientas de la ingeniería de software en el desarrollo del Módulo de Gestión Financiera y Contable del sistema ERP, bajo la metodología ágil Scrum, para automatizar el cálculo de planillas salariales, la gestión contable y el cumplimiento de la normativa tributaria SUNAT en organizaciones empresariales peruanas, entregando un producto de software funcional, documentado y verificable al finalizar el Sprint 0.

Objetivos específicos

- 1) Especificar y documentar los requisitos funcionales y no funcionales del Módulo de Gestión Financiera y Contable mediante la aplicación de técnicas de ingeniería de requisitos (entrevistas a stakeholders, análisis de procesos de negocio, revisión de normativa SUNAT), gestionando el backlog del producto bajo la metodología Scrum y generando los artefactos de documentación técnica (diagramas UML, matrices de requisitos, casos de uso) que sirvan como línea base para el desarrollo del Sprint 1.

- 2) Diseñar e implementar la arquitectura del sistema siguiendo el patrón de capas (presentación WinForms, lógica de negocio en C#/.NET Core y acceso a datos en PostgreSQL 16), documentando el diseño mediante diagramas UML (clases, secuencia, componentes, estados, actividad) por cada rol del equipo, aplicando la normalización de base de datos hasta la Forma Boyce-Codd (BCNF) para garantizar la integridad referencial y la eficiencia de las consultas, y estableciendo los procedimientos almacenados en PL/pgSQL para las operaciones críticas de cálculo de planillas y generación de reportes contables.

Visión del Producto

El Sistema ERP - Módulo de Gestión Financiera y Contable es una aplicación de escritorio desarrollada en C# con Windows Forms (WinForms) que automatiza integralmente los procesos de planillas salariales y contabilidad tributaria de una organización empresarial peruana. La aplicación se estructura en cuatro componentes funcionales interconectados, descritos a continuación.

Módulo de Planillas

El módulo de planillas constituye el subsistema encargado de la gestión automatizada de la remuneración del personal. Sus funcionalidades principales son:

- Registro de empleados: Permite al administrador de RRHH registrar, editar y consultar los datos del personal activo, incluyendo nombres completos, número de documento de identidad (DNI), cargo ocupacional, salario base mensual, fecha de ingreso, tipo de régimen pensionario (AFP o ONP) y datos bancarios para el depósito de haberes.
- Cálculo automático de planilla mensual: Ejecuta el procesamiento de la planilla para un período determinado, calculando para cada empleado:
 - Salario bruto a partir del salario base más horas extras.
 - Descuento AFP (10 % del salario bruto) si el empleado está afiliado al Sistema Privado de Pensiones.
 - Descuento ONP (13 % del salario bruto) si el empleado está afiliado al Sistema Nacional de Pensiones.

- Compensación por Tiempo de Servicios (CTS) depositada semestralmente.
- Gratificaciones legales de julio y diciembre equivalentes a 1/6 del salario bruto cada una.
- Salario neto resultante de restar los descuentos al salario bruto.
- Generación de boletas de pago en PDF: Produce documentos PDF individuales por empleado con el detalle de haberes, descuentos y neto a pagar, listos para ser distribuidos electrónicamente o impresos.
- Cálculo de CTS: Aplica la fórmula legal peruana que considera el salario bruto más la gratificación proporcional dividido entre 12 meses.
- Cálculo de gratificaciones: Aplica el beneficio legal de julio y diciembre equivalente a 1/6 del salario bruto mensual.

Módulo Contable SUNAT

El módulo contable SUNAT gestiona el registro y la generación de los libros contables electrónicos exigidos por la administración tributaria peruana. Sus funcionalidades principales son:

- Registro de comprobantes de compra y venta: Permite al contador registrar facturas, boletas de venta, notas de crédito, notas de débito, guías de remisión y recibos por honorarios. Cada comprobante incluye: tipo de comprobante, serie y número, fecha de emisión, RUC del emisor o receptor, razón social, base imponible, monto del IGV (18 %) y total.
- Cálculo automático del IGV: Al ingresar la base imponible de un comprobante, el sistema calcula automáticamente el IGV aplicando la tasa del 18 % vigente en Perú para operaciones gravadas. La fórmula aplicada es: $IGV = \text{base_imponible} \times 0.18$.
- Generación del Libro de Compras electrónico: Compila todos los comprobantes de compra registrados en un período, estructurándolos según el formato exigido por SUNAT para el Registro de Compras Electrónico, incluyendo columnas de base imponible, IGV y total.
- Generación del Libro de Ventas electrónico: Compila todos los comprobantes de venta registrados en un período, estructurándolos según el formato exigido por SUNAT para el Registro de Ventas Electrónico, incluyendo la discriminación del IGV y las operaciones gravadas, exoneradas e inafectas.

- Aplicación de la normativa tributaria vigente: El sistema mantiene tablas de configuración actualizables con las tasas impositivas (IGV 18 %, Impuesto a la Renta), los códigos de tributos SUNAT y los formatos de libros electrónicos, permitiendo adaptarse a cambios normativos sin modificar el código fuente.

Dashboard de Reportes Financieros

El dashboard de reportes financieros proporciona al gerente financiero una visión consolidada de la situación económica de la organización. Sus funcionalidades son:

- Balance general: Reporte que muestra la situación financiera en una fecha determinada, presentando los activos, pasivos y patrimonio neto de la organización.
- Estado de resultados: Reporte que muestra los ingresos, costos y gastos durante un período, determinando la utilidad o pérdida neta del ejercicio.
- Reporte de planilla consolidada: Vista agregada de todos los empleados con totales por concepto (bruto, descuentos, neto) para un período seleccionado.
- Exportación a formatos PDF y Excel: Todos los reportes pueden exportarse a PDF para distribución y a Excel para análisis adicional.
- Filtros por período y departamento: Los reportes pueden filtrarse por rango de fechas y por unidad organizativa.

Autenticación y Control de Acceso

El sistema implementa un mecanismo de autenticación basado en JSON Web Tokens (JWT) con control de acceso por roles (Role-Based Access Control, RBAC). Las funcionalidades incluyen:

- Inicio de sesión: Verificación de credenciales (usuario y contraseña) contra la base de datos, con almacenamiento seguro mediante hash BCrypt.
- Generación de token JWT: Tras la autenticación exitosa, el sistema genera un token JWT firmado con una clave secreta del servidor, que incluye los claims del usuario (ID, nombre, rol) y una fecha de expiración configurable.
- Validación de token: Cada solicitud a la API backend incluye el token JWT en el encabezado Authorization, que es validado por un middleware antes de procesar la solicitud.

- Roles y permisos: El sistema define cuatro roles preconfigurados: Administrador RRHH (acceso al módulo de planillas), Contador (acceso al módulo contable SUNAT), Gerente Financiero (acceso a reportes financieros) y Administrador del Sistema (acceso total incluyendo gestión de usuarios y roles).

Modelo de Sistema

El modelo del sistema describe la interacción entre los departamentos de la organización, los actores que representan a cada departamento y las funcionalidades que el sistema ERP proporciona a cada uno.

Departamento de Recursos Humanos

Elemento	Descripción
Departamento	Recursos Humanos (RRHH)
Actor	Administrador RRHH
Responsabilidad	Gestión integral del personal y procesamiento de planillas
Funcionalidades del sistema	El sistema automatiza el cálculo de planillas mensuales, el registro de empleados, la gestión de descuentos previsionales (AFP/ONP), el cálculo de CTS y gratificaciones, y la generación de boletas de pago en PDF

El Administrador RRHH interactúa con el sistema a través del módulo de Planillas. El flujo de trabajo inicia con el registro de un nuevo empleado, continúa con la configuración de su régimen pensionario y finaliza con la ejecución del cálculo mensual de la planilla. El sistema procesa automáticamente las reglas de negocio: aplica el descuento AFP (10 %) u ONP (13 %) según la elección del empleado, calcula la CTS según la fórmula legal, determina la gratificación proporcional y genera el reporte final.

Departamento de Contabilidad

Elemento	Descripción
Departamento	Contabilidad
Actor	Contador
Responsabilidad	Gestión de comprobantes, libros contables y cumplimiento tributario

Funcionalidades del sistema	El sistema permite registrar comprobantes de compra y venta, calcular automáticamente el IGV (18 %), generar el Libro de Compras y Libro de Ventas electrónicos en formato SUNAT, y mantener el registro histórico de operaciones contables
------------------------------------	---

El Contador utiliza el módulo Contable SUNAT para registrar los comprobantes diarios. Al ingresar un comprobante de compra o venta, el sistema valida el RUC contra la base de datos de proveedores/clientes, calcula el IGV automáticamente y almacena la transacción. Al cierre del período, el Contador ejecuta la generación del Libro de Compras y Libro de Ventas, que el sistema estructura según el formato electrónico exigido por SUNAT, incluyendo la discriminación de operaciones gravadas, exoneradas e inafectas.

Gerencia

Elemento	Descripción
Departamento	Gerencia
Actor	Gerente Financiero
Responsabilidad	Toma de decisiones estratégicas basada en información financiera
Funcionalidades del sistema	El sistema proporciona dashboards y reportes financieros (balance general, estado de resultados, planilla consolidada) con capacidades de filtrado por período y exportación a PDF/Excel

El Gerente Financiero accede al dashboard principal donde visualiza indicadores clave de desempeño (KPIs): total de planilla del mes, costo laboral total, ingresos por ventas, gastos administrativos, margen de utilidad, entre otros. Los reportes se actualizan con los datos registrados por RRHH y Contabilidad, proporcionando una vista unificada de la salud financiera de la organización. El Gerente puede profundizar en los detalles de cada reporte aplicando filtros por período, departamento o tipo de operación.

Departamento de TI/Sistemas

Elemento	Descripción
Departamento	TI / Sistemas
Actor	Administrador del Sistema
Responsabilidad	Gestión de usuarios, roles, seguridad y mantenimiento del sistema
Funcionalidades del sistema	El sistema permite crear, modificar y desactivar cuentas de usuario, asignar roles (RRHH, Contador, Gerente, Administrador), gestionar permisos, visualizar logs de acceso y actividad, y realizar tareas de mantenimiento de la base de datos

El Administrador del Sistema es el responsable de la configuración inicial del sistema: creación de la estructura organizativa, definición de usuarios y asignación de roles. También gestiona los aspectos técnicos como la configuración de la conexión a la base de datos, la programación de backups, la revisión de logs de errores y la actualización de las tasas tributarias cuando la normativa SUNAT cambia.

Variables del Sistema

El sistema ERP maneja un conjunto de variables cuantificables que determinan los resultados de los cálculos de planillas y contabilidad. A continuación se presentan las variables y sus fórmulas asociadas.

Variables del Módulo de Planillas

Salario Bruto

El salario bruto representa la remuneración total del empleado antes de aplicar descuentos. Se calcula a partir del salario base más las horas extras trabajadas en el período.

Donde:

- `salario_base`: Remuneración fija mensual del empleado (en soles).
- `horas_extras`: Número de horas extraordinarias trabajadas en el mes.
- `tarifa_hora`: Valor de la hora ordinaria, calculado como $\text{salario_base} / 240$ (considerando 30 días $\times$ 8 horas diarias = 240 horas)

mensuales). Las horas extras tienen un recargo del 25 % para las primeras dos horas y del 35 % para las siguientes, según el régimen laboral peruano.

Ejemplo: Un empleado con salario base de S/ 2,800.00 que trabaja 4 horas extras en el mes (con tarifa hora de S/ 11.67) tendrá:

- $\text{salario_bruto} = 2,800.00 + 4 \times 11.67 = 2,800.00 + 46.68 = \text{S/ } 2,846.68$

AFP (Administradora de Fondos de Pensiones)

Si el empleado está afiliado al Sistema Privado de Pensiones (AFP), el descuento mensual es del 10 % del salario bruto.

Adicionalmente, las AFP cobran una comisión sobre el salario (variable según la AFP: aproximadamente 1.5 % a 1.8 %) y una prima de seguro (aproximadamente 1.3 %). Para simplificar, el sistema considera el 10 % como descuento previsional base.

Ejemplo: Sobre un salario bruto de S/ 2,846.68:

- $\text{AFP} = 2,846.68 \times 0.10 = \text{S/ } 284.67$

ONP (Oficina de Normalización Previsional)

Si el empleado está afiliado al Sistema Nacional de Pensiones (ONP), el descuento mensual es del 13 % del salario bruto.

Ejemplo: Sobre un salario bruto de S/ 2,846.68:

- $\text{ONP} = 2,846.68 \times 0.13 = \text{S/ } 370.07$

CTS (Compensación por Tiempo de Servicios)

La CTS es un beneficio social que se deposita semestralmente (mayo y noviembre) en una entidad financiera elegida por el trabajador. El cálculo considera el salario bruto más la gratificación proporcional dividido entre 12 meses, multiplicado por los meses trabajados en el semestre.

Donde:

- **gratif_proporcional:** Es la sexta parte de la última gratificación recibida, calculada como $\text{salario_bruto} / 6$, dividida entre 12 meses.
- **meses_trabajados:** Número de meses completos trabajados en el semestre (máximo 6).

Ejemplo: Para un empleado con salario bruto de S/ 2,846.68 trabajando el semestre completo (6 meses):

- $\text{gratif_proporcional} = 2,846.68 / 6 = 474.45$

- $CTS \text{ mensual} = (2,846.68 + 474.45) / 12 = 332.13 / 12$
- $CTS \text{ semestral} = 276.76 \times 6 = S/ 1,660.56$

Gratificación

Las gratificaciones legales son beneficios que se otorgan en julio y diciembre. Equivalen a 1/6 del salario bruto mensual.

Ejemplo: Sobre un salario bruto de S/ 2,846.68:

- $Gratificación = 2,846.68 / 6 = S/ 474.45$

Descuentos Totales

Los descuentos totales son la suma de todos los descuentos aplicables al empleado en el período.

Donde quinta_categoria es la retención del Impuesto a la Renta de quinta categoría, que se aplica cuando el ingreso bruto anual proyectado supera las 7 Unidades Impositivas Tributarias (UIT).

Salario Neto

El salario neto es la cantidad que efectivamente recibe el empleado después de aplicar todos los descuentos.

Ejemplo: Para un empleado con salario bruto de S/ 2,846.68 y AFP de S/ 284.67:

- $Neto = 2,846.68 - 284.67 = S/ 2,562.01$

Variables del Módulo Contable

IGV (Impuesto General a las Ventas)

El IGV es un impuesto indirecto que grava las operaciones de venta de bienes y prestación de servicios en Perú. La tasa vigente es del 18 % (16 % de IGV + 2 % de IPM).

Donde base_imponible es el monto de la operación gravada, sin incluir el IGV.

Ejemplo: Para una factura de venta con base imponible de S/ 1,000.00:

- $IGV = 1,000.00 \times 0.18 = S/ 180.00$
- $Total = 1,000.00 + 180.00 = S/ 1,180.00$

Indicadores de Desempeño

Indicador	Fórmula	Objetivo
Tiempo de cálculo de planilla	Tiempo total para 100 empleados	< 30 segundos

Precisión del cálculo	$(\text{Cálculos correctos} / \text{Total cálculos}) \times 100 \%$	100 %
Disponibilidad del sistema	$(\text{Horas operativas} / \text{Horas totales}) \times 100 \%$	> 99.5 %
Tiempo de generación de libros SUNAT	Tiempo total para 1,000 comprobantes	< 60 segundos

Planificación - Sprint 0

Roles del equipo Scrum

De acuerdo con la metodología ágil Scrum definida por Schwaber y Sutherland (2020), el equipo se organiza en los siguientes roles:

Rol Scrum	Integrante	Responsabilidades
Product Owner	M.Sc. Juan Carlos Pinto Larico (Docente)	Define la visión del producto, prioriza el backlog, valida los entregables
Sprint / Analista de Requisitos	Alvis Danival Quispe Huayta	Levanta y documenta requisitos funcionales y no funcionales, realiza entrevistas con stakeholders, redacta historias de usuario, gestiona el backlog
Frontend Developer	Luis Fernando Cutiri Vilca	Diseña e implementa la interfaz de usuario en WinForms, aplica patrones MVVM, construye formularios, dashboards y reportes visuales
Backend Developer	Elias Ronald Ticona Callata	Implementa la lógica de negocio en C#/.NET Core, desarrolla controladores REST, servicios, autenticación JWT, integración con base de datos
Arquitecto y Admin. Base de Datos	Edison David Supo Cruz	Diseña el esquema de base de datos en

		PostgreSQL, aplica normalización 3FN/BCNF, crea procedimientos almacenados PL/pgSQL, gestiona migraciones
--	--	---

Backlog inicial del producto

A diferencia de una simple lista de características, el siguiente backlog describe el flujo funcional paso a paso que sigue el usuario al interactuar con la aplicación. Cada historia de usuario representa una secuencia completa de acciones y respuestas del sistema.

Historia 1: Autenticación y acceso al sistema

Como usuario del sistema

Quiero autenticarme con credenciales seguras

Para acceder a los módulos según mi rol asignado

Flujo funcional:

1. El usuario abre la aplicación WinForms y visualiza la pantalla de Login con campos de usuario y contraseña.
2. El usuario ingresa su nombre de usuario y contraseña, y hace clic en "Iniciar sesión".
3. El sistema envía una solicitud POST al endpoint /api/auth/login del backend con las credenciales.
4. El backend recibe la solicitud, busca el usuario en la tabla USUARIO de PostgreSQL y verifica la contraseña contra el hash almacenado (BCrypt).
5. Si las credenciales son válidas, el backend genera un token JWT firmado con los claims del usuario (ID, nombre, rol, fecha de expiración) y lo retorna en la respuesta HTTP 200.
6. El frontend almacena el token JWT en memoria (variable de sesión) y lo incluye en el encabezado Authorization: Bearer <token> para todas las solicitudes posteriores.
7. El sistema redirige al usuario al Dashboard principal, que muestra los módulos disponibles según su rol (Planillas, Contabilidad, Reportes, Administración).

8. Si las credenciales son inválidas, el sistema muestra un mensaje de error "Usuario o contraseña incorrectos" y permanece en la pantalla de Login.

Historia 2: Registro de empleados

Como Administrador RRHH

Quiero registrar los datos de un nuevo empleado

Para mantener actualizada la nómina del personal

Flujo funcional:

1. El Administrador RRHH inicia sesión con su usuario y rol (autenticación JWT).
2. El sistema muestra el Dashboard con los módulos disponibles. El Administrador hace clic en "Planillas".
3. El sistema muestra el menú del módulo Planillas con opciones: Empleados, Cálculo, Reportes.
4. El Administrador selecciona "Empleados" y el sistema muestra la lista de empleados registrados con opciones de búsqueda y filtrado.
5. El Administrador hace clic en "Nuevo Empleado" y el sistema presenta un formulario con campos: DNI, nombres, apellidos, fecha de nacimiento, fecha de ingreso, cargo, salario base, tipo de descuento (AFP/ONP), banco y número de cuenta.
6. El Administrador completa los campos y hace clic en "Guardar".
7. El frontend envía una solicitud POST al endpoint /api/empleados con los datos del empleado en formato JSON, incluyendo el token JWT en el encabezado.
8. El backend valida los datos (DNI único, formato correcto, salario > 0), inserta el registro en la tabla EMPLEADO de PostgreSQL y retorna el empleado creado con su ID.
9. El sistema muestra un mensaje de confirmación "Empleado registrado exitosamente" y actualiza la lista de empleados.
10. Si algún dato es inválido, el sistema muestra mensajes de error específicos por campo.

Historia 3: Cálculo de planilla mensual

Como Administrador RRHH

Quiero ejecutar el cálculo automático de la planilla mensual

Para obtener los montos de haberes, descuentos y neto de todos los empleados

Flujo funcional:

1. El Administrador RRHH navega al módulo Planillas y selecciona "Cálculo de Planilla".
2. El sistema muestra la pantalla de cálculo con un selector de período (mes y año), una lista de empleados activos y un botón "Calcular".
3. El Administrador selecciona el período (por ejemplo, "Mayo 2026") y hace clic en "Calcular".
4. El frontend envía una solicitud POST al endpoint `/api/planilla/calcular` con el período seleccionado.
5. El backend recibe la solicitud y ejecuta el procedimiento almacenado `sp_calcular_planilla(periodo)` en PostgreSQL.
6. El procedimiento almacenado realiza las siguientes operaciones:
 - Consulta todos los empleados activos.
 - Para cada empleado, calcula $\text{salario_bruto} = \text{salario_base} + \text{horas_extras} \times \text{tarifa_hora}$.
 - Aplica AFP (10 %) u ONP (13 %) según el tipo de descuento del empleado.
 - Calcula gratificación proporcional $= \text{salario_bruto} / 6$.
 - Calcula CTS mensual $= (\text{salario_bruto} + \text{gratif_proporcional}) / 12$.
 - Calcula descuentos totales $= \text{AFP_o_ONP}$.
 - Calcula neto $= \text{salario_bruto} - \text{descuentos}$.
 - Inserta los resultados en la tabla PLANILLA y DETALLE_PLANILLA.
1. El backend retorna al frontend una lista JSON con los resultados de cada empleado: nombre, salario_bruto, afp, onp, cts, gratificacion, descuentos, neto.
2. El frontend muestra una tabla con los resultados, incluyendo totales al pie (bruto total, descuentos totales, neto total).
3. El Administrador puede hacer clic en "Exportar PDF" para generar las boletas de pago individuales.
4. El sistema procesa la exportación usando una librería de generación de PDF (por ejemplo, iTextSharp o QuestPDF) y descarga un archivo ZIP con las boletas individuales.

Historia 4: Registro de comprobante contable

Como Contador

Quiero registrar un comprobante de compra o venta

Para mantener actualizado el registro contable y generar los libros SUNAT

Flujo funcional:

1. El Contador inicia sesión y accede al módulo Contable SUNAT.
2. El sistema muestra el menú con opciones: Comprobantes, Libro Compras, Libro Ventas, Reportes.
3. El Contador selecciona "Nuevo Comprobante".
4. El sistema presenta un formulario con campos: tipo (Factura, Boleta, Nota de Crédito, etc.), serie, número, fecha de emisión, tipo de documento del emisor/receptor (RUC, DNI), número de documento, razón social, base imponible, tipo de operación (gravada, exonerada, inafecta).
5. El Contador completa los datos y el sistema calcula automáticamente el IGV al 18 % cuando la base imponible corresponde a una operación gravada.
6. El Contador hace clic en "Guardar".
7. El sistema valida el RUC contra el formato SUNAT (11 dígitos), verifica que no exista un comprobante duplicado (misma serie y número) y guarda el registro en la tabla COMPROBANTE.
8. El sistema muestra el comprobante registrado y actualiza la lista de comprobantes del período.

Historia 5: Generación de Libro de Compras y Ventas

Como Contador

Quiero generar el Libro de Compras y el Libro de Ventas del período

Para cumplir con las obligaciones tributarias ante SUNAT

Flujo funcional:

1. El Contador navega al módulo Contable SUNAT y selecciona "Generar Libros".
2. El sistema muestra una pantalla con dos opciones: "Generar Libro de Compras" y "Generar Libro de Ventas", cada una con un selector de período.

3. El Contador selecciona el período y hace clic en "Generar Libro de Ventas".
4. El sistema consulta todos los comprobantes de venta del período seleccionado que tengan estado "Registrado".
5. El sistema estructura los datos según el formato SUNAT: período, RUC, tipo comprobante, serie, número, fecha, base imponible gravada, base imponible exonerada, IGV, total.
6. El sistema agrupa los comprobantes por tipo (facturas, boletas, notas de crédito, notas de débito) y calcula los totales por grupo.
7. El sistema muestra una vista previa del Libro de Ventas en pantalla, con los totales al pie (base imponible total, IGV total, total general).
8. El Contador puede exportar el libro a formato PDF o Excel, o enviarlo directamente desde el sistema a SUNAT mediante el servicio web de Libros Electrónicos (opcional).
9. El mismo flujo aplica para el Libro de Compras, pero consultando los comprobantes de compra.

Historia 6: Visualización de reportes financieros

Como Gerente Financiero

Quiero visualizar el balance general y el estado de resultados

Para tomar decisiones estratégicas informadas

Flujo funcional:

1. El Gerente Financiero inicia sesión y el sistema lo redirige al Dashboard principal.
2. El Dashboard muestra indicadores clave: total planilla del mes, ingresos por ventas, costo de ventas, utilidad bruta, IGV por pagar, cuentas por cobrar y por pagar.
3. El Gerente selecciona "Balance General" y el sistema muestra un reporte estructurado con activos (corrientes y no corrientes), pasivos (corrientes y no corrientes) y patrimonio neto.
4. El Gerente puede cambiar el período de reporte mediante un selector de fechas.
5. El sistema actualiza dinámicamente los valores del balance según el período seleccionado, consultando las tablas contables y de planillas.

6. El Gerente puede hacer clic en "Exportar a PDF" o "Exportar a Excel" para descargar el reporte.

Análisis de Requisitos

Requisitos por Rol

A continuación se presentan los requisitos funcionales (RF) y no funcionales (RNF) específicos para cada rol del equipo, organizados según las responsabilidades de cada integrante.

Alvis Danival Quispe Huayta - Sprint / Analista de Requisitos

Requisitos Funcionales del Proceso de Análisis y Documentación:

ID	Nombre	Requisito Funcional
RF-ALV-01	Levantamiento de requisitos	El sistema de documentación debe permitir al Analista registrar actas de entrevistas con stakeholders, incluyendo fecha, entrevistado, preguntas realizadas, respuestas obtenidas y observaciones, para asegurar la trazabilidad de los requisitos desde su origen.
RF-ALV-02	Redacción de historias de usuario	El sistema debe permitir redactar historias de usuario siguiendo el formato estándar "Como [rol] quiero [acción] para [beneficio]", asignando un identificador único, prioridad (1-5), puntos de esfuerzo estimado y criterios de aceptación, para facilitar la gestión del backlog.
RF-ALV-03	Gestión del backlog del producto	El sistema debe permitir crear, priorizar, reordenar y filtrar el backlog del producto por sprint, estado (pendiente, en curso, completado, validado) y prioridad, para mantener una visión actualizada del trabajo pendiente del equipo.
RF-ALV-04	Validación de requisitos	El sistema debe permitir registrar la validación de cada requisito con el Product Owner y el equipo de desarrollo, incluyendo fecha de revisión, resultados de la validación (aprobado/rechazado/observado)

		y comentarios, para garantizar que los requisitos cumplen con las expectativas del negocio.
RF-ALV-05	Generación de documentación técnica	El sistema debe permitir generar automáticamente documentos de especificación de requisitos (SRS) a partir del backlog, incluyendo tabla de contenido, descripción de actores, matriz de requisitos funcionales y no funcionales, y diagramas UML embebidos, para producir la documentación del Sprint 0 de forma consistente.

Requisitos No Funcionales del Proceso de Análisis y Documentación:

ID	Nombre	Categoría	Criterio Medible	Requisito No Funcional
RNF-ALV-01	Trazabilidad de requisitos	Mantenibilidad	Cada requisito debe tener trazabilidad bidireccional	El sistema debe mantener trazabilidad bidireccional entre cada requisito y su origen (entrevista, normativa, reunión) para que el 100 % de los requisitos puedan ser rastreados hasta su fuente original.
RNF-ALV-02	Formato estandarizado	Usabilidad	Documentos generados en formato SRS estándar IEEE 830	La documentación generada debe cumplir con el estándar IEEE 830-1998 para especificaciones de requisitos de software, incluyendo

				todas las secciones obligatorias.
RNF-ALV-03	Consistencia de requisitos	Calidad	Sin conflictos entre requisitos	El sistema debe detectar y alertar sobre requisitos funcionales conflictivos o duplicados durante el registro, verificando contra la base de conocimiento de requisitos existentes.
RNF-ALV-04	Actualización en tiempo real	Rendimiento	Propagación de cambios < 5 segundos	Los cambios realizados en el backlog deben propagarse a todos los artefactos de documentación asociados en menos de 5 segundos desde su guardado.
RNF-ALV-05	Acceso basado en roles	Seguridad	Solo usuarios autorizados pueden modificar requisitos	El sistema debe restringir la modificación de requisitos a los usuarios con rol de Analista o Product Owner, permitiendo solo lectura al resto del equipo.

Luis Fernando Cutiri Vilca - Frontend Developer

Requisitos Funcionales de la Interfaz WinForms:

ID	Nombre	Requisito Funcional
RF-LUI-01	Navegación por	La interfaz WinForms

	módulos	debe proporcionar un menú de navegación lateral que permita al usuario acceder a los módulos de Planillas, Contabilidad SUNAT, Reportes Financieros y Administración, mostrando únicamente los módulos para los cuales el usuario tiene permisos según su rol.
RF-LUI-02	Formularios de entrada de datos	La interfaz debe presentar formularios con validación en tiempo real para el registro de empleados, comprobantes y configuraciones, mostrando mensajes de error específicos por campo (formato inválido, valor fuera de rango, campo obligatorio vacío) antes de enviar los datos al servidor.
RF-LUI-03	Visualización de tablas y resultados	La interfaz debe mostrar los resultados de cálculos (planillas, libros contables) en tablas con formato profesional, incluyendo alternancia de colores en filas, ordenamiento por columna, filtros por período y paginación, con totales calculados al pie de cada columna numérica.
RF-LUI-04	Dashboard de indicadores financieros	La interfaz debe presentar un dashboard con indicadores clave (KPIs) en tarjetas visuales, incluyendo total de planilla, ingresos, gastos, IGV

		por pagar y utilidad, con gráficos de barras o circulares generados dinámicamente a partir de los datos del backend.
RF-LUI-05	Exportación de reportes	La interfaz debe proporcionar botones de exportación en todas las vistas de reportes, permitiendo al usuario descargar los datos en formato PDF (con diseño profesional) y Excel (con formato tabular), invocando los servicios correspondientes del backend.

Requisitos No Funcionales de la Interfaz WinForms:

ID	Nombre	Categoría	Criterio Medible	Requisito No Funcional
RNF-LUI-01	Tiempo de carga inicial	Rendimiento	Pantalla principal visible en ≤ 3 segundos	La interfaz WinForms debe cargar la pantalla principal (Dashboard) en un tiempo máximo de 3 segundos desde la ejecución del ejecutable, excluyendo el tiempo de autenticación.
RNF-LUI-02	Capacidad de respuesta de la interfaz	Rendimiento	Tiempo de respuesta a interacciones < 200 ms	La interfaz debe responder a las interacciones del usuario (clic en botones,

				navegación, cambios de selección) en menos de 200 milisegundos, sin congelamiento de la ventana principal.
RNF-LUI-03	Diseño responsivo adaptativo	Usabilidad	Interfaz funcional en resoluciones 1280×720 a 1920×1080	La interfaz debe adaptarse correctamente a resoluciones de pantalla desde 1280×720 píxeles hasta 1920×1080 píxeles, manteniendo la legibilidad de textos y la disposición de controles.
RNF-LUI-04	Consistencia visual	Usabilidad	Misma paleta de colores, tipografía y espaciado en todas las pantallas	Todas las pantallas de la aplicación deben utilizar una paleta de colores consistente (colores institucionales azul #1F5C99 y grises), la misma familia tipográfica (Segoe UI) y espaciados uniformes.
RNF-LUI-05	Accesibilidad	Usabilidad	Soporte para navegación por teclado y lectores de pantalla	La interfaz debe ser navegable completamente mediante teclado (tab, enter, flechas)

				y los controles deben tener etiquetas accesibles para lectores de pantalla conforme a las directrices WCAG 2.1 nivel AA.
--	--	--	--	--

Elias Ronald Ticona Callata - Backend Developer

Requisitos Funcionales de la API ASP.NET Core:

ID	Nombre	Requisito Funcional
RF-ELI-01	Autenticación JWT	El backend debe implementar un endpoint POST /api/auth/login que reciba credenciales (usuario y contraseña), las valide contra la base de datos PostgreSQL y retorne un token JWT firmado con los claims del usuario (ID, nombre, rol) y una expiración configurable (por defecto 1 hora).
RF-ELI-02	CRUD de empleados	El backend debe implementar endpoints RESTful para el api/empleados: GET (listar todos los empleados activos, con filtros por nombre y cargo), GET/{id} (obtener empleado por ID), POST (crear nuevo empleado), PUT/{id} (actualizar datos del empleado) y DELETE/{id} (desactivar empleado, ocultación lógica).
RF-ELI-03	Cálculo de planilla	El backend debe implementar un endpoint POST /api/planilla/calcular que reciba un período (mes y año), ejecute el procedimiento almacenado PostgreSQL sp_calcular_planilla para todos los empleados activos y

		retorne los resultados estructurados en una lista JSON con los campos: id_empleado, nombre, salario_bruto, afp, onp, cts, gratificacion, descuentos, neto.
RF-ELI-04	Registro y consulta de comprobantes	El backend debe implementar endpoints RESTful para api/comprobantes: POST (registrar comprobante con validación de RUC y cálculo automático de IGV), GET (listar comprobantes con filtros por tipo, período, RUC) y GET/{id} (obtener detalle completo del comprobante incluyendo IGV y base imponible).
RF-ELI-05	Generación de libros SUNAT	El backend debe implementar endpoints GET /api/libros/compras?periodo=X y GET /api/libros/ventas?periodo=X que consulten los comprobantes del período, estructuren los datos según el formato SUNAT de libros electrónicos y los retornen en formato JSON, incluyendo totales por grupo de comprobante y totales generales.

Requisitos No Funcionales de la API ASP.NET Core:

ID	Nombre	Categoría	Criterio Medible	Requisito No Funcional
RNF-ELI-01	Tiempo de respuesta	Rendimiento	API responde en ≤ 500 ms para el 95 % de las solicitudes	El backend debe procesar y responder a las solicitudes HTTP en un tiempo máximo de 500 milisegundos

				para el 95 % de las peticiones, excluyendo el tiempo de red.
RNF-ELI-02	Seguridad de endpoints	Seguridad	Todos los endpoints protegidos por JWT	Todos los endpoints de la API, excepto /api/auth/login, deben requerir un token JWT válido en el encabezado Authorization; el middleware debe rechazar solicitudes sin token o con token expirado con código 401.
RNF-ELI-03	Manejo de errores	Mantenibilidad	Respuestas de error con código y mensaje descriptivo	El backend debe manejar todas las excepciones y retornar respuestas HTTP estructuradas con código de estado (400, 404, 500, etc.), un mensaje descriptivo del error y un identificador único de correlación para trazabilidad.
RNF-ELI-04	Capacidad de concurrencia	Rendimiento	Soporte para 50 usuarios concurrentes	El backend debe soportar al menos 50 usuarios

				concurrentes realizando solicitudes simultáneas, con un tiempo de respuesta promedio que no exceda los 800 milisegundos bajo esta carga.
RNF-ELI-05	Documentación de API	Mantenibilidad	API documentada con Swagger/OpenAPI	El backend debe exponer la documentación interactiva de la API mediante Swagger UI en la ruta /swagger, incluyendo descripción de cada endpoint, parámetros, tipos de datos, códigos de respuesta y ejemplos de solicitudes.

Edison David Supo Cruz - Arquitecto y Admin. Base de Datos

Requisitos Funcionales del Diseño y Gestión de la BD PostgreSQL:

ID	Nombre	Requisito Funcional
RF-EDI-01	Esquema normalizado hasta BCNF	La base de datos debe implementar un esquema relacional normalizado hasta la Forma Boyce-Codd (BCNF), garantizando que toda dependencia funcional no trivial tenga una superclave como determinante, para

		eliminar redundancias y anomalías de actualización.
RF-EDI-02	Procedimientos almacenados para cálculos	La base de datos debe incluir procedimientos almacenados en PL/pgSQL para las operaciones críticas: sp_calcular_planilla(periodo) que calcule la planilla de todos los empleados activos en una sola transacción, y sp_generar_libro(tipo, periodo) que estructure los comprobantes según formato SUNAT.
RF-EDI-03	Integridad referencial con FK y constraints	La base de datos debe implementar claves foráneas (FK) con acciones de integridad referencial (ON DELETE RESTRICT, ON UPDATE CASCADE), restricciones CHECK para validación de datos (salario > 0, IGV >= 0, tipo_comprobante IN ('Factura','Boleta','NC','ND')), y valores por defecto WHERE aplicable.
RF-EDI-04	Índices para optimización de consultas	La base de datos debe crear índices en columnas de búsqueda frecuente (periodo, ruc, fecha, username, tipo_comprobante, id_empleado) y en las columnas de las claves foráneas, utilizando índices compuestos WHERE las consultas involucran múltiples columnas de filtrado.
RF-EDI-05	Gestión de usuarios y roles de BD	La base de datos debe implementar roles de PostgreSQL (rol_app_read, rol_app_write, rol_app_admin) con privilegios mínimos necesarios (principio de

		mínimo privilegio), asignando permisos específicos sobre esquemas, tablas, secuencias y funciones, y denegando explícitamente el acceso a tablas sensibles.
--	--	---

Requisitos No Funcionales del Diseño y Gestión de la BD PostgreSQL:

ID	Nombre	Categoría	Criterio Medible	Requisito No Funcional
RN F-EDI-01	Tiempo de ejecución de cálculos	Rendimiento	Procedimiento sp_calcular_planilla ejecuta en ≤ 30 s para 100 empleados	El procedimiento almacenado sp_calcular_planilla debe completar el cálculo de planilla para 100 empleados en un tiempo máximo de 30 segundos, incluyendo todas las operaciones de inserción, actualización y cálculo.
RN F-EDI-02	Disponibilidad de la base de datos	Fiabilidad	Disponibilidad del 99.5 % en horario laboral	La base de datos debe estar disponible el 99.5 % del tiempo durante el horario laboral (lunes a viernes de 8:00 a 18:00), con planes de respaldo y recuperación ante fallos.
RN F-EDI-03	Seguridad de datos financieros	Seguridad	Datos cifrados en reposo y en tránsito	La base de datos debe implementar cifrado en tránsito mediante SSL/TLS para

				todas las conexiones desde la aplicación, y cifrado en reposo (transparent data encryption) para tablas que contengan datos sensibles (salarios, contraseñas, RUC).
RN F-EDI-04	Capacidad de almacenamiento	Escalabilidad	Soporte para 5 años de datos operativos	La base de datos debe estar diseñada para almacenar y consultar eficientemente al menos 5 años de datos operativos (planillas mensuales, comprobantes diarios), con estrategias de particionamiento por período y archivado histórico.
RN F-EDI-05	Migraciones automatizadas	Mantenibilidad	Cambios de esquema gestionados con EF Core Migrations	Todos los cambios al esquema de la base de datos deben gestionarse mediante migraciones de Entity Framework Core, con scripts de migración versionados en el repositorio Git y procedimientos de rollback

				documentados.
--	--	--	--	---------------

Matriz consolidada de Requisitos Funcionales

ID	Nombre	Actor	Prioridad	Sprint
RF-001	Gestión de Empleados (CRUD)	Administrador RRHH	Alta	Sprint 1
RF-002	Cálculo de Planillas	Administrador RRHH	Alta	Sprint 1
RF-003	Reporte de Pago (PDF)	Administrador RRHH	Alta	Sprint 1
RF-004	Registro de Comprobantes	Contador	Alta	Sprint 2
RF-005	Libro de Compras y Ventas	Contador	Alta	Sprint 2
RF-006	Normativa Tributaria SUNAT	Contador	Alta	Sprint 2
RF-007	Reportes Financieros	Gerente Financiero	Media	Sprint 3
RF-008	Autenticación JWT	Administrador Sistema	Alta	Sprint 1

Matriz consolidada de Requisitos No Funcionales

ID	Categoría	Criterio Medible	Requisito No Funcional
RNF-001	Rendimiento	Cálculo planilla ≤ 30 s / 100 empleados	El sistema debe calcular la planilla de 100 empleados con tiempo de respuesta máximo de 30 segundos.
RNF-002	Seguridad	Cifrado AES-256, autenticación JWT	El sistema debe cifrar credenciales y datos financieros con AES-256 y autenticación JWT con control de roles.

RNF-003	Usabilidad	Tareas frecuentes en $\leq$ 3 clics	La interfaz debe permitir completar tareas frecuentes (registrar empleado, calcular planilla, ver reporte) en no más de 3 interacciones.
RNF-004	Compatibilidad	Windows 10+, .NET 8	El sistema debe operar en Windows 10 y superiores con .NET 8 Runtime instalado.
RNF-005	Mantenibilidad	Arquitectura modular con módulos independientes	La arquitectura debe ser modular, permitiendo actualizar o reemplazar un módulo (planillas, contable, reportes) sin afectar los demás.

Diagramas UML

Esta sección presenta los diagramas UML correspondientes a cada integrante del equipo, describiendo en texto los elementos principales de cada diagrama. Los diagramas han sido generados programáticamente utilizando la biblioteca Matplotlib de Python y se incluyen como referencias visuales en los anexos del documento.

Alvis Danival Quispe Huayta - Sprint / Analista de Requisitos

Diagrama de Casos de Uso - Proceso de Análisis de Requisitos

Actores:

- Analista de Requisitos: Actor principal que realiza el levantamiento y gestión de requisitos.

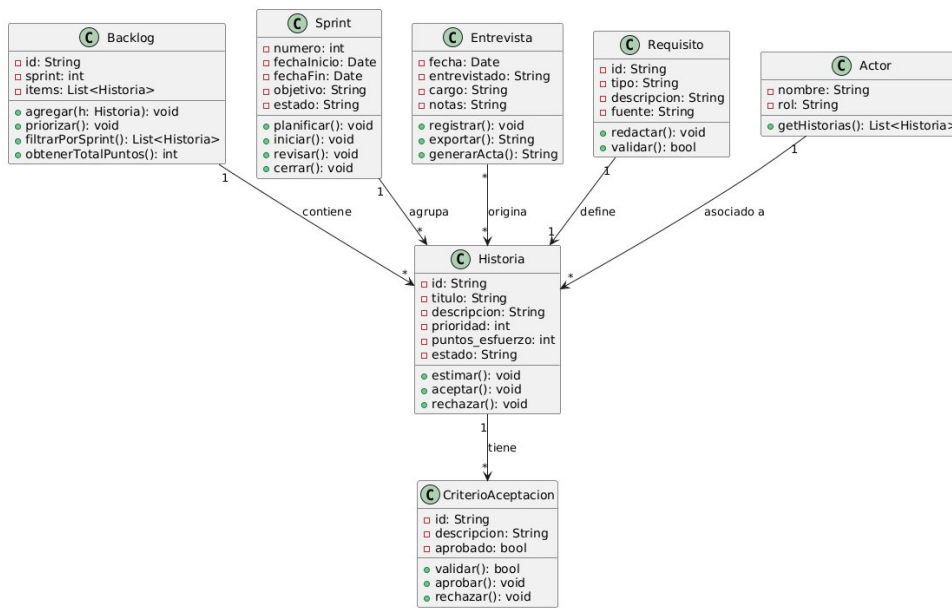
- Product Owner: Actor secundario que valida y prioriza los requisitos.
- Equipo de Desarrollo: Actor secundario que revisa y estima las historias de usuario.

Casos de Uso:

1. Levantar requisitos de negocio: El Analista realiza entrevistas con stakeholders para identificar necesidades funcionales del sistema. Incluye la elaboración de actas de entrevista y la identificación de procesos de negocio.
2. Redactar historias de usuario: El Analista documenta cada requisito identificado como una historia de usuario siguiendo el formato estándar "Como [rol] quiero [acción] para [beneficio]", asignando identificador único y criterios de aceptación.
3. Definir criterios de aceptación: Para cada historia de usuario, el Analista define las condiciones específicas que deben cumplirse para considerar la funcionalidad como completa y correcta.
4. Gestionar backlog del producto: El Analista mantiene el backlog priorizado, actualizando el estado de cada historia (pendiente, en curso, completada, validada) según el avance del sprint.
5. Planificar Sprint 0: El Analista coordina con el Product Owner y el equipo la planificación del Sprint 0, definiendo el alcance, los objetivos y las tareas necesarias.
6. Revisar y validar requisitos: El Analista presenta los requisitos al equipo de desarrollo para validación técnica, asegurando que sean factibles técnicamente y consistentes entre sí.
7. Priorizar historias: El Analista trabaja con el Product Owner para priorizar las historias según el valor de negocio, la urgencia y las dependencias entre historias.

Diagrama de Clases - Artefactos del Sprint

Alvis - Diagrama de Clases
Artefactos del Sprint



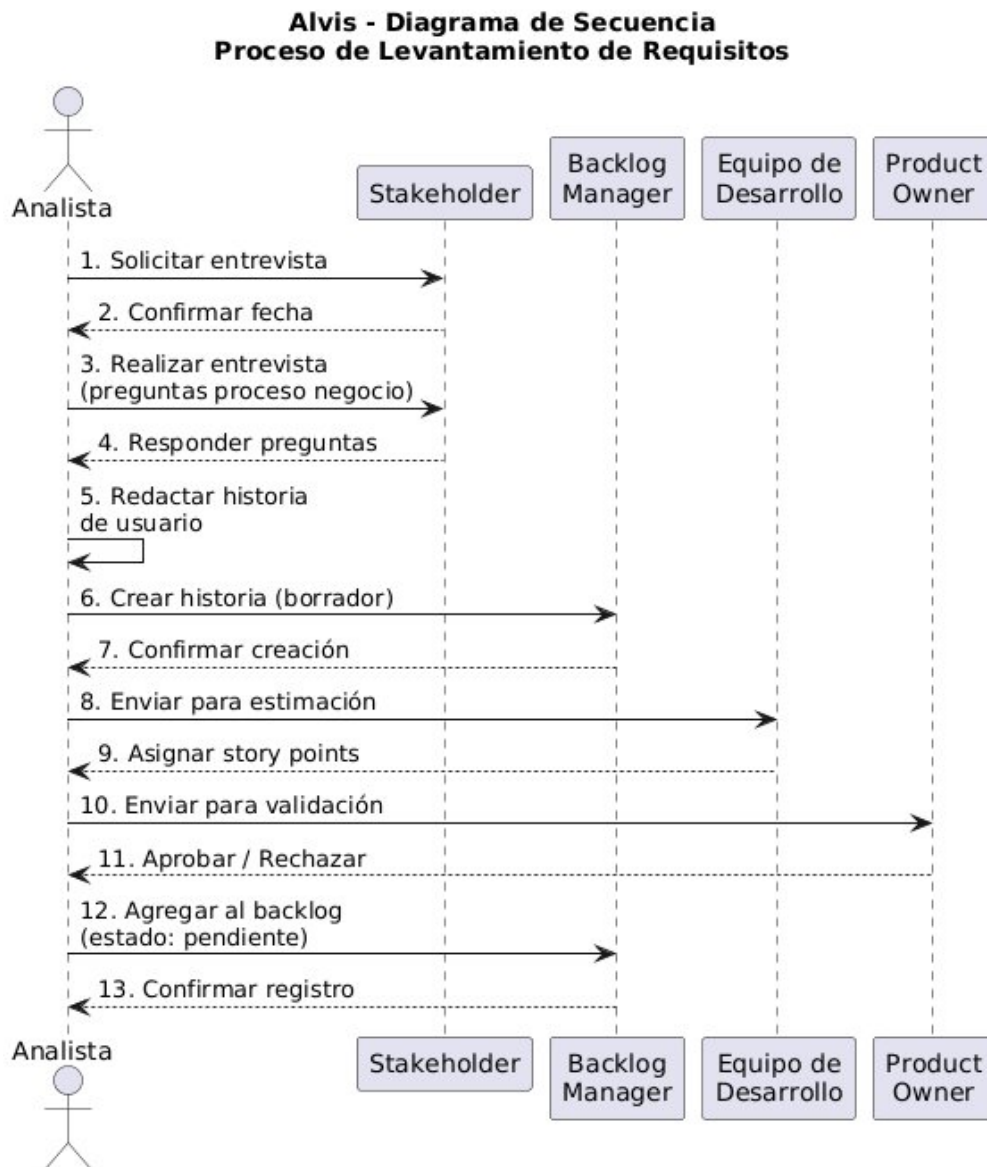
Clases del modelo de dominio del proceso de análisis:

Clase	Atributos	Métodos
Historia	- id: String - titulo: String - descripcion: String - prioridad: int - puntos_esfuerzo: int - estado: String - fecha_creacion: Date	estimar() aceptar() rechazar() obtenerCriterios()
Backlog	- id: String - sprint: int - items: List<Historia> - fecha_actualizacion: Date	agregar(h: Historia) priorizar() filtrarPorSprint() obtenerTotalPuntos()
Sprint	- numero: int - fechaInicio: Date - fechaFin: Date - objetivo: String - estado: String	planificar() iniciar() revisar() cerrar()
CriterioAceptacion	- id: String - descripcion: String - aprobado: bool - historia_id: String	validar() aprobar() rechazar()
Entrevista	- fecha: Date - entrevistado: String - cargo: String - notas: String - duracion_minutos: int	registrar() exportar() generarActa()
Requisito	- id: String - tipo: String (funcional/no funcional) - descripcion: String - historia: Historia - fuente: String	redactar() validar() asociarHistoria()
Actor	- nombre: String - rol: String - descripcion: String	getHistorias() getCasosUso()
Rol	- nombre: String - permisos: List<String>	getPermisos() tieneAcceso(modulo)

Relaciones:

- Backlog (1) → (N) Historia: un backlog contiene múltiples historias de usuario.
- Sprint (1) → (N) Historia: un sprint agrupa varias historias a desarrollar.
- Historia (1) → (N) CriterioAceptacion: cada historia tiene uno o más criterios de aceptación.
- Entrevista (N) → (N) Historia: una entrevista puede originar múltiples historias, y una historia puede originarse de múltiples entrevistas.
- Requisito (1) → (1) Historia: cada requisito se expresa como una historia de usuario.
- Actor (1) → (N) Historia: un actor puede tener múltiples historias asociadas.

Diagrama de Secuencia - Proceso de Levantamiento de Requisitos



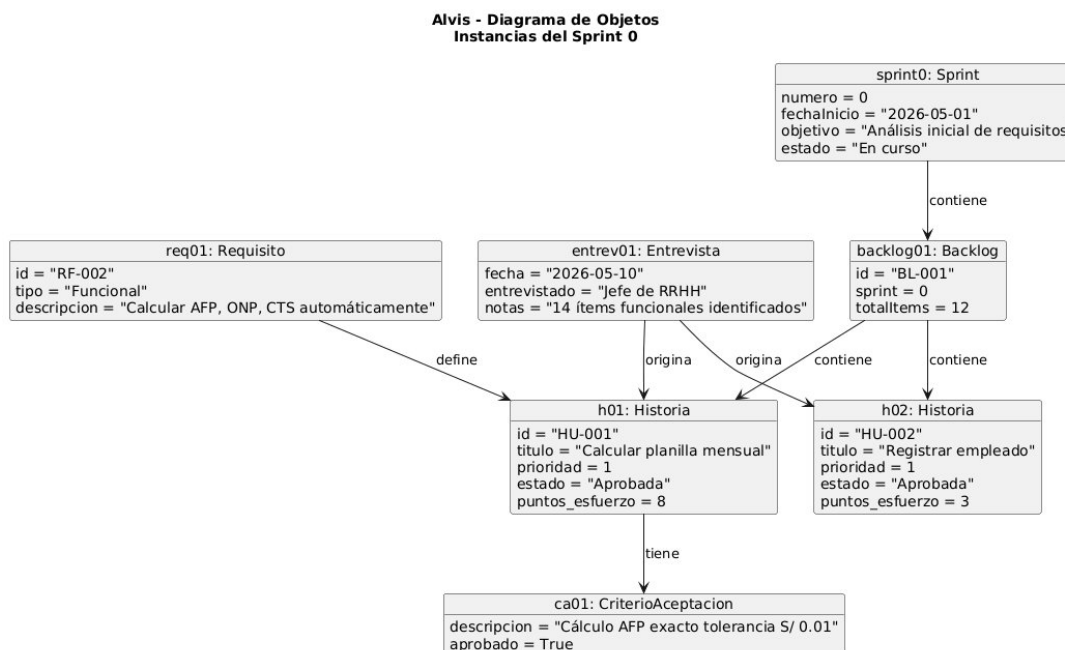
Escenario: El Analista de Requisitos realiza el proceso completo de levantamiento de una historia de usuario, desde la entrevista inicial hasta su inclusión en el backlog.

Secuencia de mensajes:

1. El Analista solicita una entrevista al Stakeholder, coordinando fecha y hora.
2. El Stakeholder confirma la fecha de la entrevista.
3. El Analista realiza la entrevista con una serie de preguntas predefinidas sobre los procesos de negocio.

4. El Stakeholder responde las preguntas, proporcionando información detallada sobre sus necesidades y flujos de trabajo actuales.
5. El Analista redacta la historia de usuario con la información obtenida, incluyendo criterios de aceptación preliminares.
6. El sistema de backlog (Backlog Manager) confirma la creación de la historia como borrador (estado "borrador").
7. El Analista envía la historia al Equipo de Desarrollo para estimación técnica.
8. El equipo revisa la historia y asigna puntos de esfuerzo (story points) usando la escala de Fibonacci (1, 2, 3, 5, 8, 13).
9. El Analista envía la historia estimada al Product Owner para validación final.
10. El Product Owner revisa la historia, verifica que cumple con la visión del producto y la aprueba o rechaza.
11. La historia aprobada se agrega al backlog con estado "pendiente" y prioridad asignada.

Diagrama de Objetos - Instancias del Sprint 0



Instancias del modelo durante el Sprint 0:

Objeto	Clase	Atributos
sprint0	Sprint	numero=0, fechaInicio="2026-05-01", objetivo="Análisis inicial de requisitos",

		estado="En curso"
backlog01	Backlog	id="BL-001", sprint=0, totalItems=12
h01	Historia	i d = " H U - 0 0 1 " , titulo="Calcular planilla mensual", prioridad=1, estado="Aprobada", puntos_esfuerzo=8
h02	Historia	i d = " H U - 0 0 2 " , titulo="Registrar empleado", prioridad=1, estado="Aprobada", puntos_esfuerzo=3
ca01	CriterioAceptacion	id="CA-001", desc="El cálculo AFP debe ser exacto con tolerancia S/ 0.01", aprobado=True
entrev01	Entrevista	fecha="2026-05-10", entrevistado="Jefe de RRHH", notas="14 ítems funcionales identificados"
req01	Requisito	i d = " R F - 0 0 2 " , tipo="Funcional", desc="Sistema debe calcular AFP, ONP, CTS automáticamente"

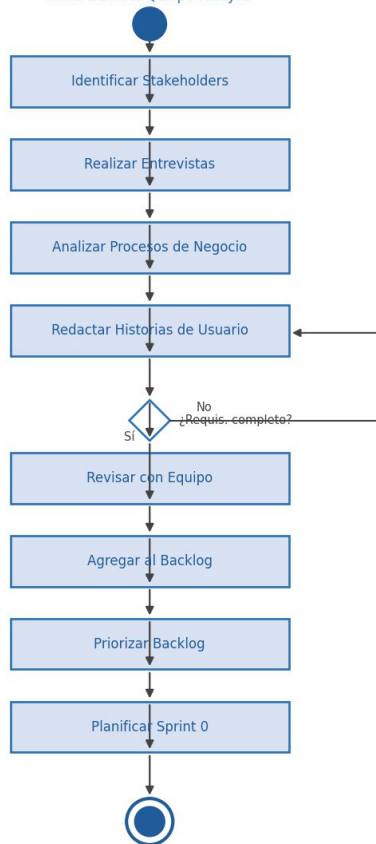
Relaciones entre instancias:

- sprint0 → backlog01 (pertenece a)
- backlog01 → h01, h02 (contiene)
- h01 → ca01 (tiene criterio de aceptación)
- entrev01 → h01, h02 (origina)
- req01 → h01 (define formalmente)

Diagrama de Actividad - Proceso Sprint 0

Diagrama de Actividad - Proceso Sprint 0 (Análisis)

Alvis Danival Quispe Huayta



Descripción del flujo de actividades del proceso de análisis:

El diagrama de actividad modela el flujo de trabajo completo del Sprint 0 desde la perspectiva del Analista de Requisitos.

1. Inicio: El proceso comienza con la identificación de los stakeholders del proyecto.
2. Identificar Stakeholders: Actividad donde se determina quiénes son las personas clave que serán entrevistadas.
3. Realizar Entrevistas: Se ejecutan las entrevistas planificadas, documentando cada una en actas formales.
4. Analizar Procesos de Negocio: Se analizan las respuestas de las entrevistas y se modelan los procesos de negocio actuales.
5. Redactar Historias de Usuario: Se redactan las historias de usuario para cada requisito identificado.
6. Decisión: ¿Requisitos completos?: Punto de decisión donde se evalúa si se ha cubierto todo el alcance definido.

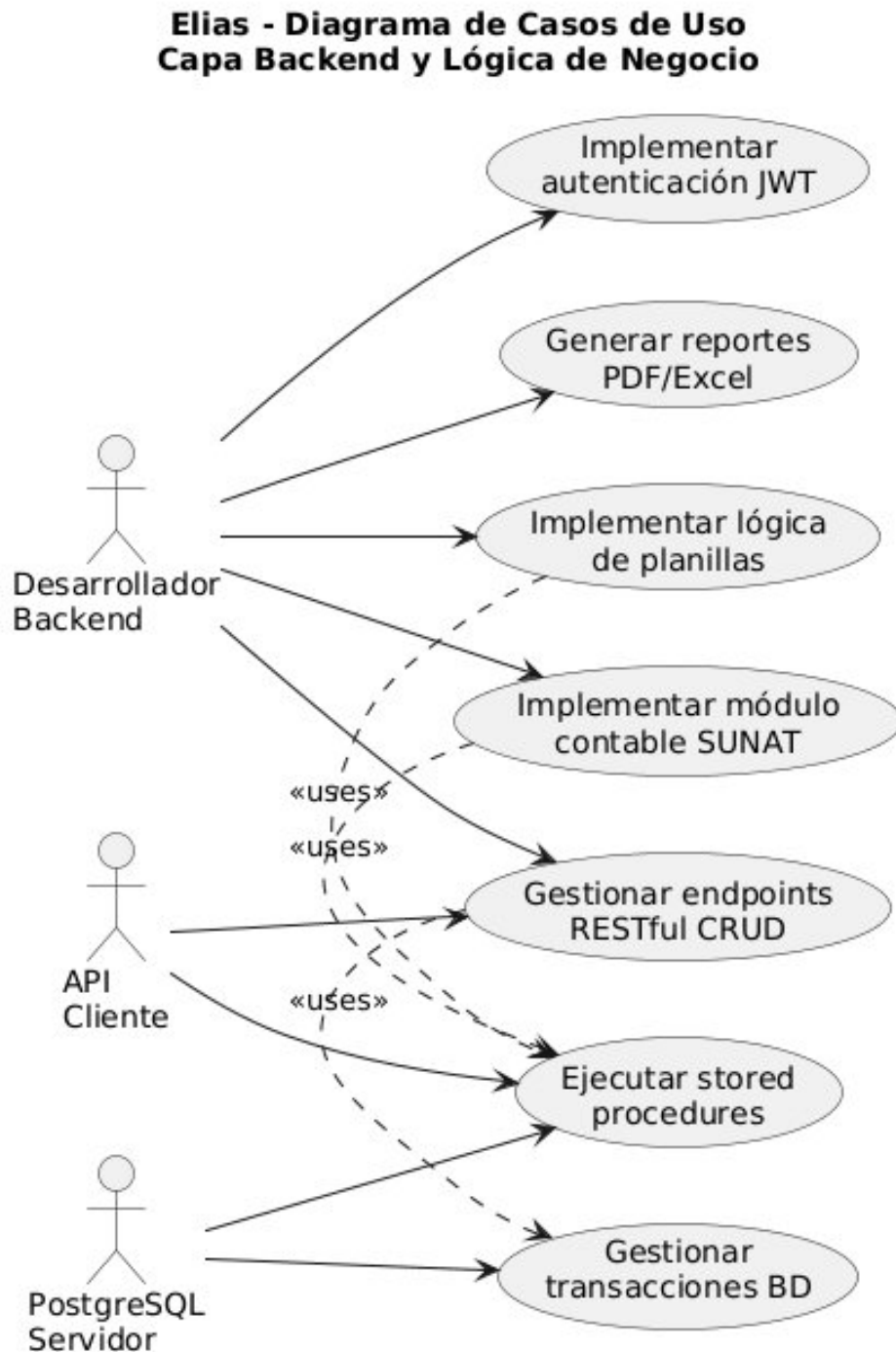
- Si No → Se retorna a "Realizar Entrevistas" para profundizar en áreas faltantes.

- Si Sí → Se continúa con la revisión.

1. Revisar con Equipo: Las historias se presentan al equipo de desarrollo para validación técnica y estimación.
2. Agregar al Backlog: Las historias validadas se incorporan al backlog del producto.
3. Priorizar Backlog: El Analista y el Product Owner priorizan las historias según valor de negocio y dependencias.
4. Planificar Sprint 0: Se define el alcance final del Sprint 0 y se asignan las tareas a los miembros del equipo.
5. Fin: El proceso concluye con el backlog listo para el inicio del desarrollo.

Elias Ronald Ticona Callata - Backend Developer

Diagrama de Casos de Uso - Capa Backend y Lógica de Negocio



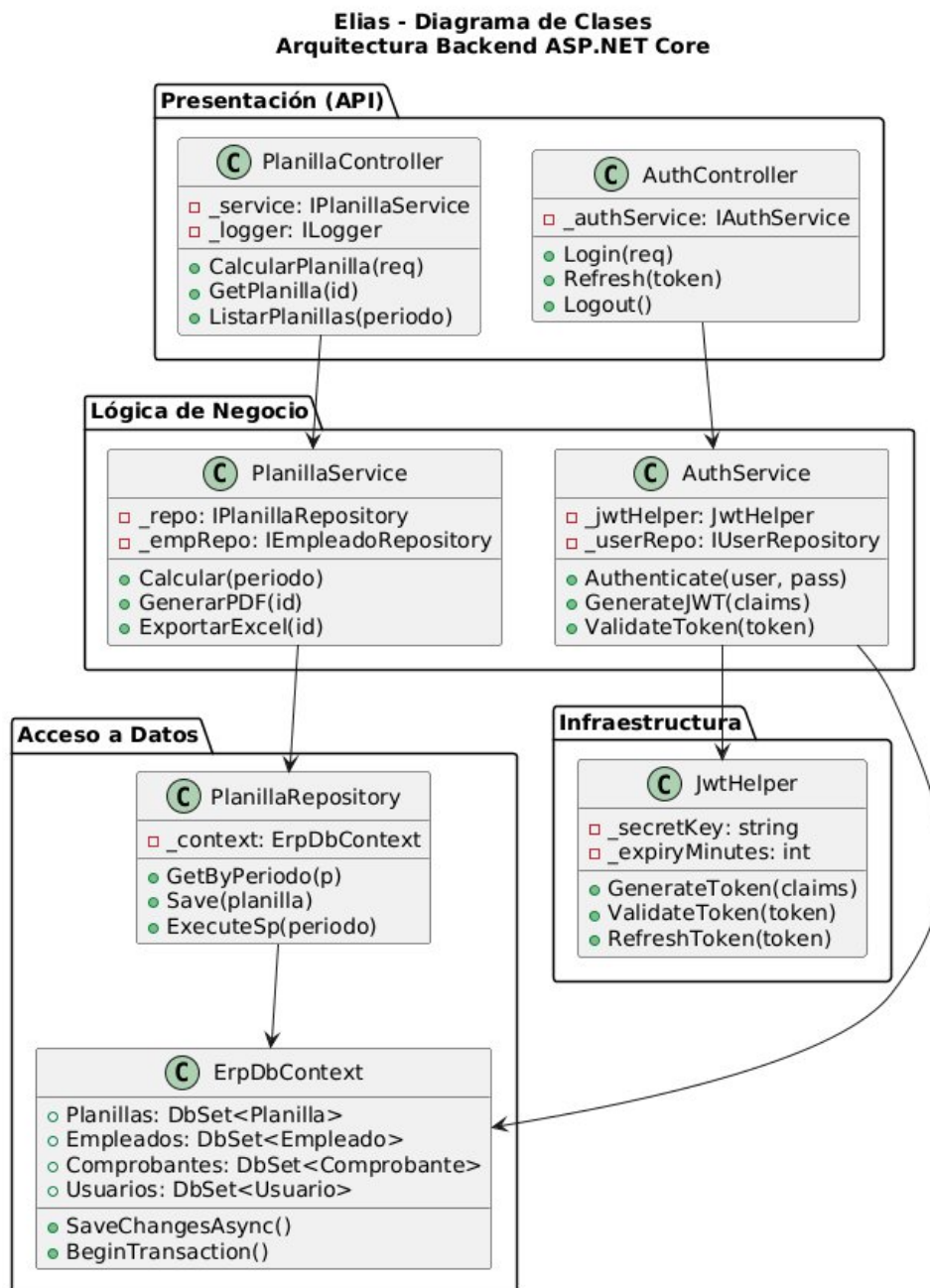
Actores:

- Desarrollador Backend: Actor principal que implementa y mantiene los servicios del backend.
- API Cliente: Actor secundario que representa a la aplicación frontend que consume los servicios REST.
- Base de Datos: Actor secundario que representa al motor PostgreSQL que ejecuta los procedimientos almacenados.

Casos de Uso:

1. Implementar autenticación JWT: El Desarrollador Backend implementa el endpoint de login que valida credenciales y genera tokens JWT firmados.
2. Gestionar endpoints RESTful: El Desarrollador implementa los controladores REST para las operaciones CRUD de empleados, comprobantes y planillas.
3. Implementar lógica de planillas: El Desarrollador implementa el servicio de planillas con las reglas de negocio para cálculo de AFP, ONP, CTS y gratificaciones.
4. Implementar módulo contable SUNAT: El Desarrollador implementa el servicio contable con las reglas de cálculo de IGV, generación de libros electrónicos y formatos SUNAT.
5. Generar reportes PDF/Excel: El Desarrollador implementa la generación de documentos PDF (boletas, reportes) y Excel (libros contables).
6. Ejecutar stored procedures: El API Cliente invoca los procedimientos almacenados PostgreSQL para operaciones críticas como el cálculo de planilla.
7. Gestionar transacciones BD: El API Cliente utiliza el contexto de Entity Framework Core para manejar transacciones atómicas.

Diagrama de Clases - Arquitectura Backend ASP.NET Core



Clases del backend siguiendo el patrón Controller → Service → Repository → DbContext:

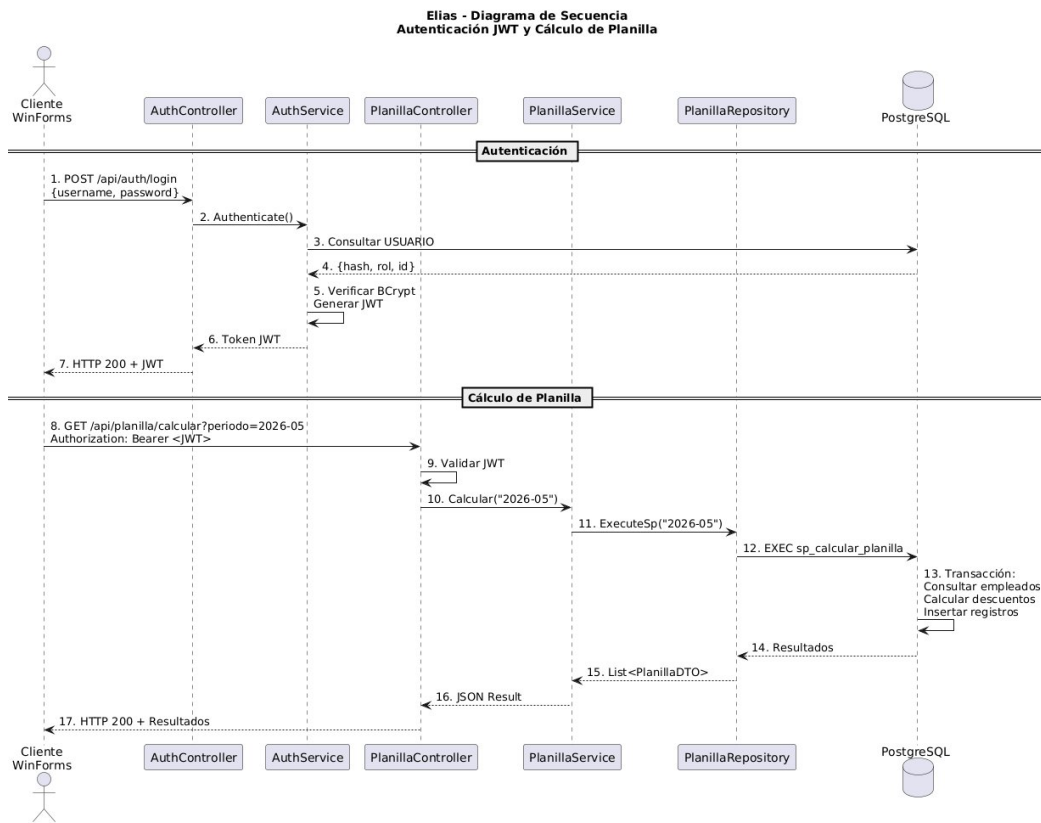
Clase	Atributos	Métodos	Capa
PlanillaController	- _service: IPlanillaService - _logger: ILogger	CalcularPlanilla(req) GetPlanilla(id) ListarPlanillas(periodo)	Presentación (API)
PlanillaService	- _repo: IPlanillaRepository - _empRepo: IEmpleadoRepository	Calcular(periodo) GenerarPDF(id) ExportarExcel(id)	Lógica de Negocio
PlanillaRepository	- _context: ErpDbContext	GetByPeriodo(p) Save(planilla) GetDetalle(id) ExecuteSp(periodo)	Acceso a Datos
AuthController	- _authService: IAuthService	Login(req) Refresh(token) Logout()	Presentación (API)
AuthService	- _jwtHelper: JwtHelper - _userRepo: IUserRepository	Authenticate(user, pass) GenerateJWT(claims) ValidateToken(token)	Lógica de Negocio
JwtHelper	- _secretKey: string - _expiryMinutes: int - _issuer: string	GenerateToken(claims) ValidateToken(token) RefreshToken(token)	Infraestructura
ErpDbContext	Planillas: DbSet<Planilla> Empleados: DbSet<Empleado> Comprobantes: DbSet<Comprobante> Usuarios: DbSet<Usuario>	SaveChangesAsync() BeginTransaction() MigrateAsync()	Acceso a Datos
Empleado (Entity)	Id: int Nombre: string Cargo: string SalarioBase:	CalcularDescuento() Validar()	Modelo de Dominio

	decimal TipoDescuento: string FechaIngreso: DateTime		
Comprobante (Entity)	Id: int Tipo: string Serie: string Numero: string Ruc: string Monto: decimal Igv: decimal Fecha: DateTime	CalcularIGV() ValidarRUC()	Modelo de Dominio

Relaciones:

- PlanillaController (1) → (1) PlanillaService: el controlador depende del servicio (inyección de dependencias).
- PlanillaService (1) → (1) PlanillaRepository: el servicio utiliza el repositorio para acceso a datos.
- PlanillaRepository (1) → (1) ErpDbContext: el repositorio utiliza el contexto de Entity Framework Core.
- AuthController (1) → (1) AuthService: dependencia mediante inyección.
- AuthService (1) → (1) JwtHelper y (1) IUserRepository: el servicio utiliza el helper JWT y el repositorio de usuarios.

Diagrama de Secuencia - Autenticación JWT y Cálculo de Planilla



Escenario integrado: El usuario se autentica mediante JWT y posteriormente ejecuta el cálculo de planilla.

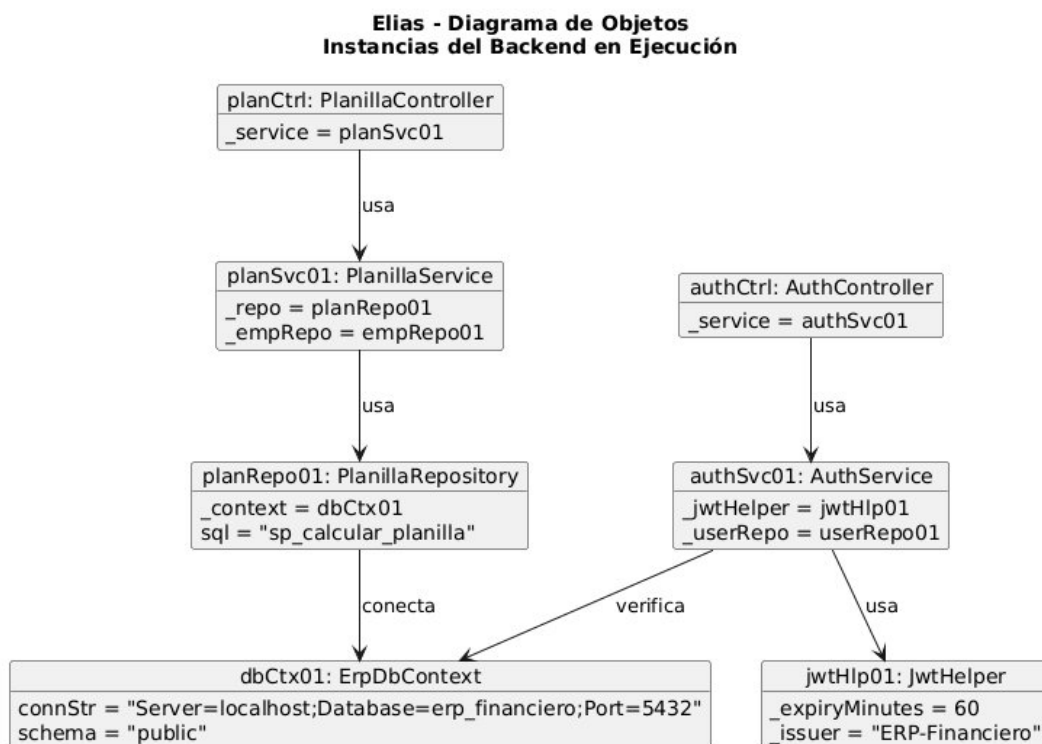
Secuencia de mensajes (Parte 1 - Autenticación):

1. El Cliente WinForms envía una solicitud POST /api/auth/login con las credenciales {username, password}.
2. AuthController recibe la solicitud y llama a AuthService.Authenticate().
3. AuthService consulta la tabla USUARIO en PostgreSQL mediante el repositorio de usuarios.
4. PostgreSQL retorna el registro del usuario con el hash de la contraseña.
5. AuthService verifica la contraseña usando BCrypt. Si es correcta, genera un token JWT con los claims del usuario.
6. AuthService retorna el token JWT al AuthController.
7. AuthController retorna una respuesta HTTP 200 OK con el token JWT y los datos básicos del usuario.

Secuencia de mensajes (Parte 2 - Cálculo de Planilla):

1. El Cliente WinForms envía una solicitud GET /api/planilla/calcular?periodo=2026-05 con el token JWT en el encabezado Authorization.
2. PlanillaController recibe la solicitud, valida el token JWT mediante el middleware de autenticación.
3. PlanillaController llama a PlanillaService.Calcular(periodo).
4. PlanillaService invoca el repositorio PlanillaRepository.ExecuteSp(periodo).
5. PlanillaRepository ejecuta el procedimiento almacenado sp_calcular_planilla('2026-05') en PostgreSQL.
6. El procedimiento almacenado procesa la planilla dentro de una transacción: consulta empleados activos, calcula descuentos, inserta registros en PLANILLA y DETALLE_PLANILLA, y retorna los resultados.
7. PostgreSQL retorna los resultados al repositorio.
8. PlanillaService estructura los resultados en una lista de objetos PlanillaDTO.
9. PlanillaController serializa los datos a JSON y retorna una respuesta HTTP 200 OK con los resultados.

Diagrama de Objetos - Instancias del Backend en Ejecución



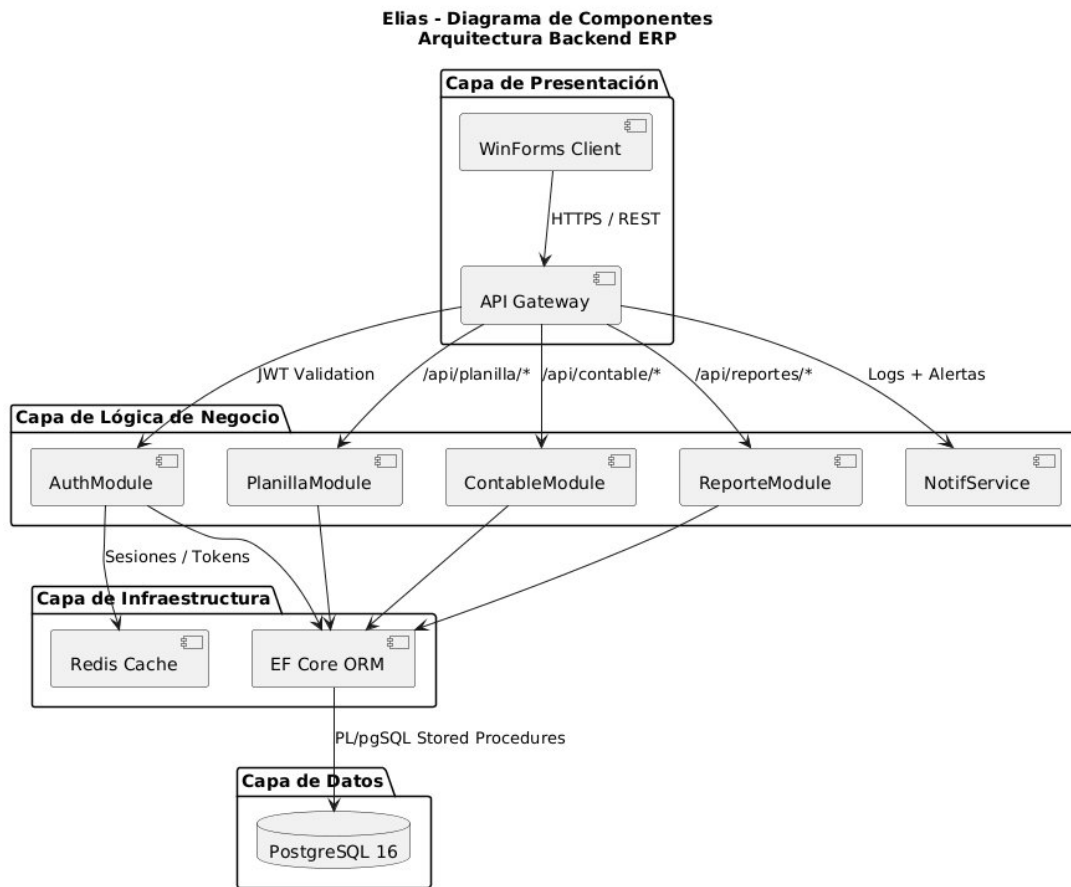
Instancias de objetos del backend durante una sesión de cálculo de planilla:

Objeto	Clase	Atributos
planCtrl	PlanillaController	_service=planSvc01 (instancia de PlanillaService), _logger=logger01
planSvc01	PlanillaService	_repo=planRepo01 (instancia de PlanillaRepository), _empRepo=empRepo01
planRepo01	PlanillaRepository	_context=dbCtx01 (instancia de ErpDbContext), sql="sp_calcular_planilla"
authCtrl	AuthService	_service=authSvc01 (instancia de AuthService)
authSvc01	AuthService	_jwtHelper=jwtHlp01 (instancia de JwtHelper), _userRepo=userRepo01
jwtHlp01	JwtHelper	_secretKey="*****" (256-bit key), _expiry=60 (minutos), _issuer="ERP-Financiero"
dbCtx01	ErpDbContext	connStr="Server=localhost;Database=erp_financiero;Port=5432;...", schema="public"

Relaciones entre instancias:

- planCtrl → planSvc01 (usa →): el controlador delega el cálculo al servicio.
- planSvc01 → planRepo01 (usa →): el servicio delega la persistencia al repositorio.
- planRepo01 → dbCtx01 (conecta →): el repositorio utiliza el contexto de EF Core.
- authCtrl → authSvc01 (usa →): el controlador de autenticación delega al servicio.
- authSvc01 → jwtHlp01 (usa →): el servicio utiliza el helper para generar/validar tokens.
- authSvc01 → dbCtx01 (verifica →): el servicio consulta usuarios en la BD.

Diagrama de Componentes - Arquitectura Backend ERP



Componentes del backend organizados en capas:

Componente	Descripción	Tecnología
API Gateway	Punto de entrada único para todas las solicitudes del frontend. Enruta las peticiones a los controladores correspondientes, aplica middleware de autenticación JWT, logs y manejo de errores.	ASP.NET Core 8 Middleware Pipeline
AuthModule	Componente encargado de la autenticación y autorización. Implementa login con verificación BCrypt, generación y validación de tokens JWT, y refresh de tokens expirados.	ASP.NET Core Identity + JWT Bearer
PlanillaModule	Componente de lógica de negocio para el cálculo de planillas. Contiene las reglas de cálculo de AFP, ONP, CTS, gratificaciones y salario neto. Invoca procedimientos almacenados PostgreSQL.	C# .NET Core Class Library
ContableModule	Componente de lógica de negocio para la contabilidad SUNAT. Gestiona comprobantes, libros electrónicos, cálculo de IGV y formatos SUNAT.	C# .NET Core Class Library
ReporteModule	Componente de generación de reportes. Produce documentos PDF (boletas de pago,	QuestPDF + EPPlus

	reportes financieros) y archivos Excel (libros contables, planillas).	
EF Core ORM	Capa de acceso a datos que abstrae las operaciones con PostgreSQL. Utiliza Entity Framework Core con el enfoque Code-First y migraciones automatizadas.	Entity Framework Core 8 + Npgsql
NotifService	Componente de notificaciones. Envía alertas por email (errores críticos, confirmaciones de cálculo) y registra eventos en archivos de log estructurados.	Serilog + SMTP Client
PostgreSQL 16	Motor de base de datos relacional. Contiene los procedimientos almacenados en PL/pgSQL para operaciones críticas, índices optimizados y esquema normalizado.	PostgreSQL 16 + PL/pgSQL
Redis Cache	Almacenamiento en caché para tokens JWT invalidados, sesiones activas y limitación de tasa (rate limiting).	StackExchange.Redis

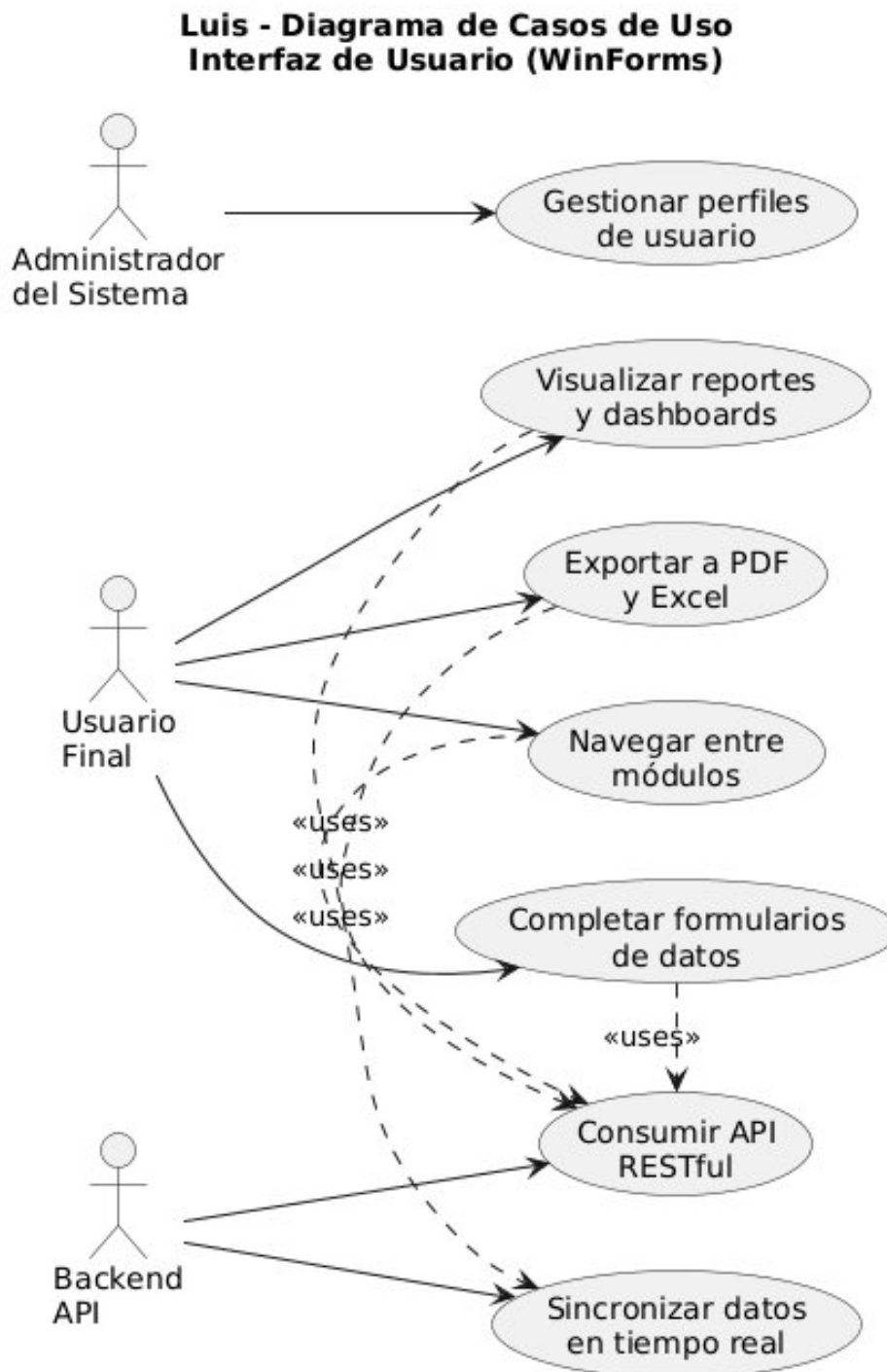
Flujo de comunicación entre componentes:

1. API Gateway recibe la solicitud HTTP del frontend.
2. El middleware de autenticación valida el token JWT (AuthModule).
3. La solicitud se enruta al controlador correspondiente (PlanillaController, ContableController).
4. El controlador invoca el módulo de negocio (PlanillaModule, ContableModule).
5. El módulo de negocio utiliza EF Core ORM para acceder a PostgreSQL.

6. Las operaciones críticas se ejecutan mediante procedimientos almacenados en PostgreSQL.
7. Los resultados se retornan al frontend como respuesta JSON.

Luis Fernando Cutiri Vilca - Frontend Developer

Diagrama de Casos de Uso - Interfaz de Usuario (WinForms)



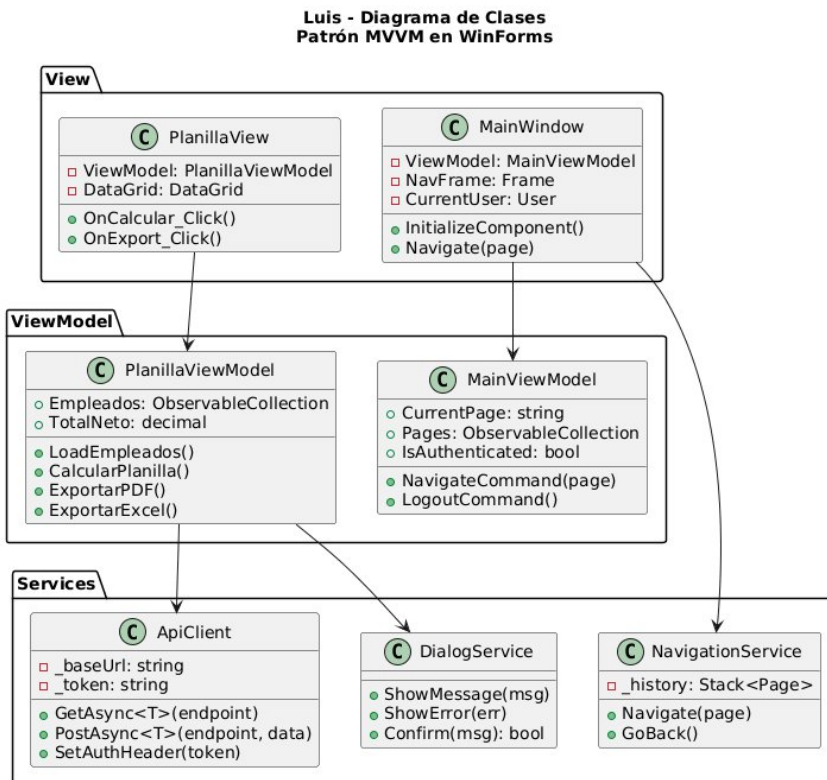
Actores:

- Usuario Final: Actor principal que interactúa con la interfaz para realizar todas las operaciones del sistema.
- Administrador del Sistema: Actor secundario que gestiona usuarios, roles y configuraciones del sistema.
- Backend API: Actor secundario que representa al servidor backend que provee los datos.

Casos de Uso:

1. Navegar entre módulos: El usuario navega entre los módulos de Planillas, Contabilidad SUNAT, Reportes Financieros y Administración mediante el menú lateral.
2. Completar formularios de datos: El usuario ingresa datos en formularios (registro de empleados, comprobantes, búsquedas) con validación en tiempo real.
3. Visualizar reportes y dashboards: El usuario visualiza reportes financieros, tablas de planillas y dashboards con indicadores clave.
4. Exportar PDF y Excel: El usuario exporta cualquier reporte o libro contable a formato PDF o Excel con un solo clic.
5. Gestionar perfiles de usuario: El Administrador del Sistema gestiona las cuentas de usuario, roles y permisos desde la interfaz de administración.
6. Consumir API RESTful: La interfaz se comunica con el backend mediante solicitudes HTTP asíncronas para obtener y enviar datos.
7. Sincronizar datos en tiempo real: La interfaz actualiza los datos mostrados cuando ocurren cambios en el backend (nuevos cálculos, registros actualizados).

Diagrama de Clases - Patrón MVVM en WinForms



Clases de la capa de presentación siguiendo el patrón Model-View-ViewModel (MVVM):

Clase	Atributos	Métodos	Tipo
MainWindow	- ViewModel: MainViewModel - NavFrame: Frame - CurrentUser: User	InitializeComponent() Navigate(page) ShowNotification(msg) OnClosing()	View
MainViewModel	CurrentPage: string Pages: ObservableCollection IsAuthenticated: bool	NavigateCommand(page) LogoutCommand() LoadModules()	ViewModel
PlanillaViewModel	Empleados: ObservableCollection<EmpleadoDTO> PeriodoSeleccionado: string TotalBruto: decimal TotalDescuentos: decimal TotalNeto: decimal	LoadEmpleados() CalcularPlanilla() ExportarPDF() ExportarExcel()	ViewModel

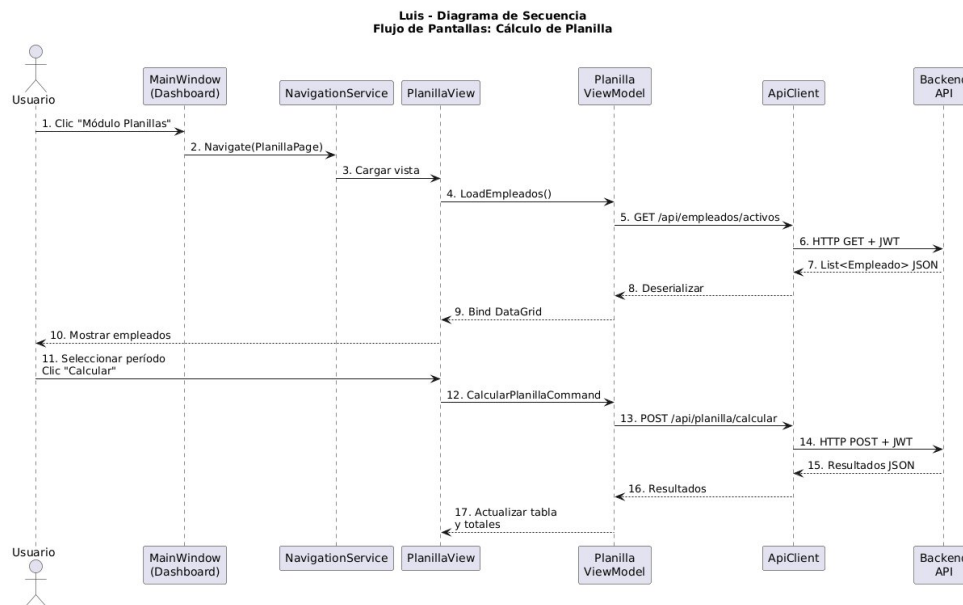
PlanillaView	- ViewModel: PlanillaViewModel - DataGrid: DataGrid - BtnCalcular: Button	InitializeComponent() BindData() OnCalcular_Click() OnExport_Click()	View
EmpleadoView Model	Nombre: string Cargo: string SalarioBase: decimal TipoDescuento: string FechaIngreso: DateTime	Save() Delete() Validate()	ViewModel
NavigationService	- _frame: Frame - _history: Stack<Page> - _current: Page	Navigate(page) GoBack() GoForward() GetCurrent()	Service
ApiClient	- _baseUrl: string - _token: string - _httpClient: HttpClient	GetAsync<T>(endpoint) PostAsync<T>(endpoint, data) PutAsync<T>(endpoint, data) DeleteAsync(endpoint) SetAuthHeader(token)	Service
DialogService	- _owner: Window	ShowMessage(msg) ShowError(err) Confirm(msg): bool ShowProgress()	Service
ICommand(RelayCommand)	- _execute: Action<object> - _canExecute: Func<object, bool>	Execute(param) CanExecute(param) RaiseCanExecuteChanged()	Infrastructure

Relaciones:

- MainWindow (1) → (1) MainViewModel: asigna el ViewModel al DataContext.
- MainWindow (1) → (1) NavigationService: utiliza el servicio de navegación.
- PlanillaView (1) → (1) PlanillaViewModel: el ViewModel es el DataContext de la View.

- PlanillaViewModel (1) → (1) ApiClient: consume la API REST del backend.
- PlanillaViewModel (1) → (1) DialogService: muestra mensajes y diálogos al usuario.
- Todas las ViewModel usan ICommand (RelayCommand) para bindear acciones desde la View.

Diagrama de Secuencia - Flujo de Pantallas: Cálculo de Planilla



Escenario: El usuario navega desde la pantalla de inicio hasta ejecutar el cálculo de planilla y visualizar los resultados.

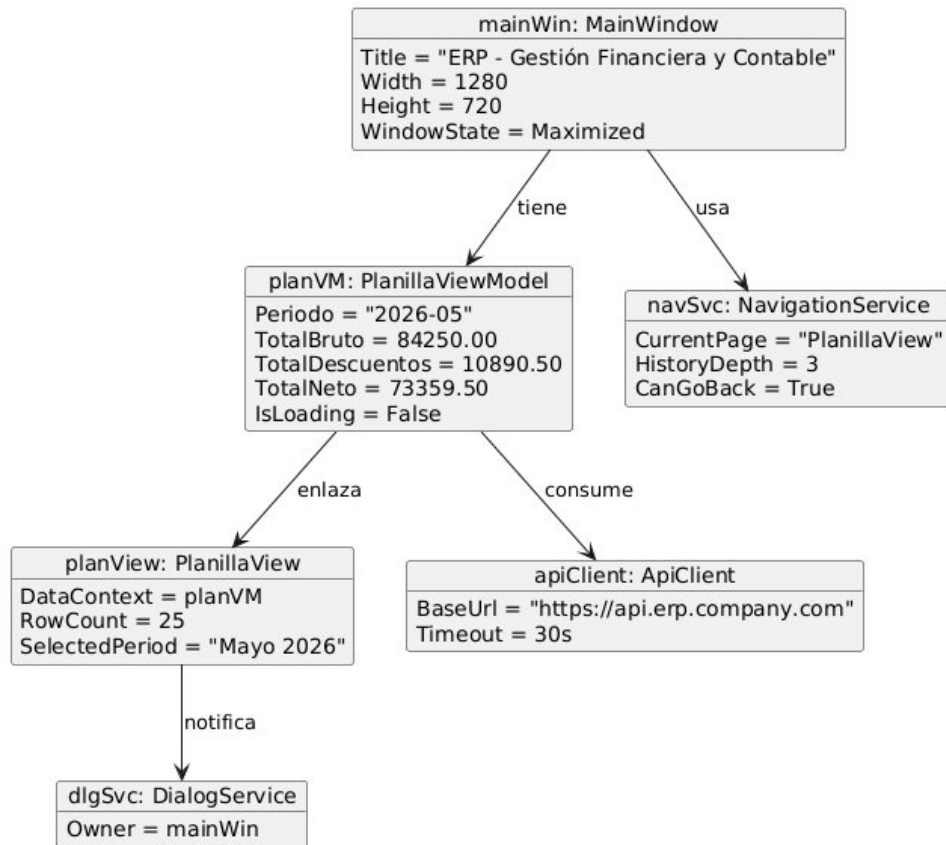
Secuencia de mensajes:

1. El usuario hace clic en "Módulo Planillas" en el Dashboard.
2. MainWindow recibe el evento y llama a `NavigationService.Navigate(PlanillaPage)`.
3. NavigationService carga PlanillaView en el Frame principal.
4. PlanillaView inicializa su PlanillaViewModel y llama a `LoadEmpleados()`.
5. PlanillaViewModel invoca `ApiClient.GetAsync<List<Empleado>>("/api/empleados/activos")`.
6. ApiClient agrega el token JWT al encabezado Authorization y envía la solicitud GET al backend.
7. El backend procesa la solicitud y retorna la lista de empleados activos en formato JSON.

8. ApiClient deserializa la respuesta JSON en una lista de objetos EmpleadoDTO.
9. PlanillaViewModel recibe los empleados y los asigna a la colección observable.
10. El DataGridView en PlanillaView se actualiza automáticamente mostrando los empleados.
11. El usuario selecciona el período (mes/año) y hace clic en "Calcular".
12. PlanillaView ejecuta el comando CalcularPlanillaCommand en el ViewModel.
13. PlanillaViewModel invoca ApiClient.PostAsync("/api/planilla/calcular", {periodo}).
14. El backend ejecuta el cálculo y retorna los resultados (lista de resultados por empleado).
15. PlanillaViewModel actualiza las propiedades (TotalBruto, TotalDescuentos, TotalNeto) con los totales calculados.
16. La View muestra la tabla de resultados y actualiza los indicadores de totales.

Diagrama de Objetos - Instancias WinForms en Tiempo de Ejecución

Luis - Diagrama de Objetos
Instancias WinForms en Tiempo de Ejecución



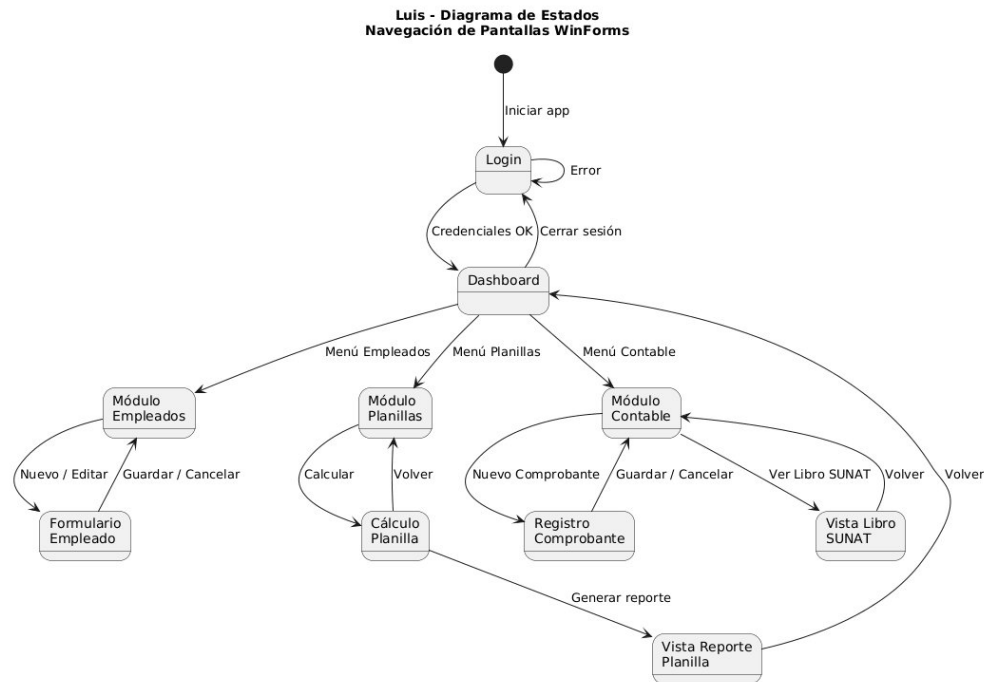
Instancias de objetos de la interfaz durante una sesión activa:

Objeto	Clase	Atributos
mainWin	MainWindow	Title="ERP - Gestión Financiera y Contable", Width=1280, Height=720, WindowState=Maximized
planVM	PlanillaViewModel	P e r i o d o = " 2 0 2 6 - 0 5 " , TotalBruto=84250.00, TotalDescuentos=10890.50, TotalNeto=73359.50, IsLoading=False
planView	PlanillaView	DataContext=planVM, RowCount=25 (empleados), SelectedPeriod="Mayo 2026"
navSvc	NavigationService	CurrentPage="PlanillaView", HistoryDepth=3, CanGoBack=True
apiClient	ApiClient	BaseUrl="https://api.erp.company.com", Token="eyJhbGciOiJIUzI1NiIs..." (JWT), Timeout=30s
dlgSvc	DialogService	Owner=mainWin, LastMessage=""

Relaciones entre instancias:

- mainWin → planVM (tiene ViewModel): la ventana principal contiene la instancia del ViewModel de planillas.
- planVM → planView (enlaza a): el ViewModel es el DataContext de la vista de planillas (binding bidireccional).
- mainWin → navSvc (usa): la ventana principal utiliza el servicio de navegación para cambiar entre páginas.
- planVM → apiClient (consume): el ViewModel utiliza el cliente API para comunicarse con el backend.
- planView → dlgSvc (notifica): la vista utiliza el servicio de diálogo para mostrar mensajes al usuario.

Diagrama de Estados - Navegación de Pantallas WinForms



Descripción de los estados de la interfaz y las transiciones entre pantallas:

Estado	Descripción	Transiciones
Login	Pantalla inicial de inicio de sesión. Muestra campos de usuario y contraseña, botón de inicio de sesión.	→ Dashboard (credenciales OK)
Dashboard	Pantalla principal con menú de módulos disponibles según el rol del usuario. Muestra tarjetas de acceso rápido.	→ Módulo Empleados → Módulo Planillas → Módulo Contable → Módulo Reportes → Login (logout)
Módulo Empleados	Lista de empleados registrados con opciones de búsqueda, filtrado, nuevo, editar y eliminar.	→ Formulario Empleado (nuevo/editar) → Dashboard (volver)
Módulo Planillas	Pantalla de cálculo de planilla con selector de período, botón calcular y tabla de resultados.	→ Cálculo Planilla (ejecutar) → Dashboard (volver)

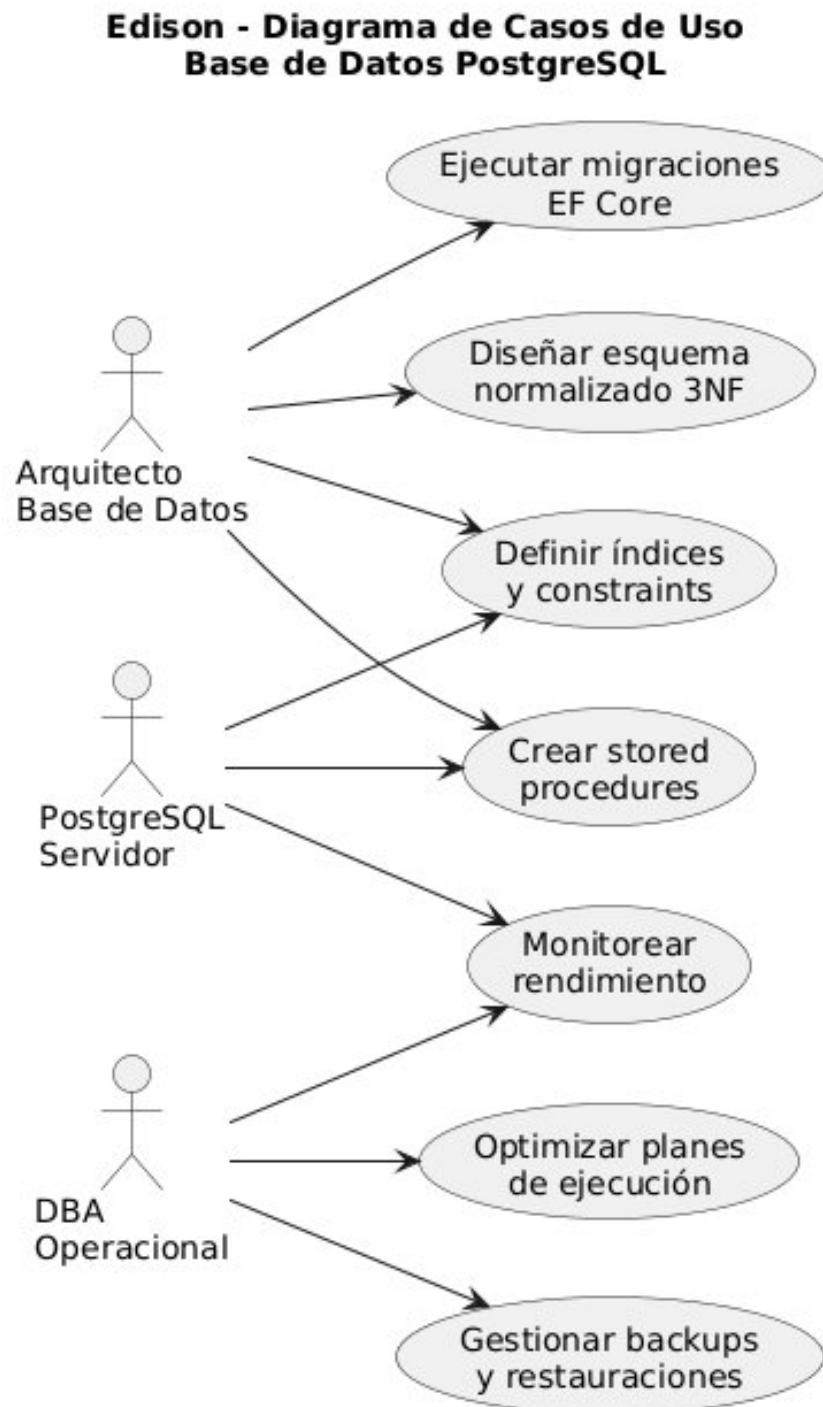
Módulo Contable	Menú del módulo contable con opciones: Comprobantes, Libro Compras, Libro Ventas.	→ Registro Comprobante → Vista Libro SUNAT → Dashboard (volver)
Formulario Empleado	Formulario de registro/edición de empleado con campos de datos personales y laborales.	→ Módulo Empleados (guardar/cancelar)
Cálculo Planilla	Vista de resultados del cálculo con tabla detallada por empleado y totales.	→ Vista Reporte (generar reporte) → Módulo Planillas (volver)
Registro Comprobante	Formulario de registro de comprobante contable con validación de RUC y cálculo automático de IGV.	→ Módulo Contable (guardar/cancelar)
Vista Reporte Planilla	Reporte PDF de boleta de pago o planilla consolidada, con opciones de exportación.	→ Dashboard (volver)
Vista Libro SUNAT	Vista del Libro de Compras o Ventas generado, con totales y opción de exportación.	→ Dashboard (volver)

Transiciones clave:

- Login → Dashboard: autenticación exitosa.
- Dashboard → Módulos y viceversa: navegación por menú.
- Cada módulo → Formularios específicos: para crear o editar registros.
- Cálculo Planilla → Vista Reporte: generación de boleta de pago.
- Desde cualquier vista de reporte → Dashboard: cierre de sesión (logout).

Edison David Supo Cruz - Arquitecto y Admin. Base de Datos

Diagrama de Casos de Uso - Base de Datos PostgreSQL



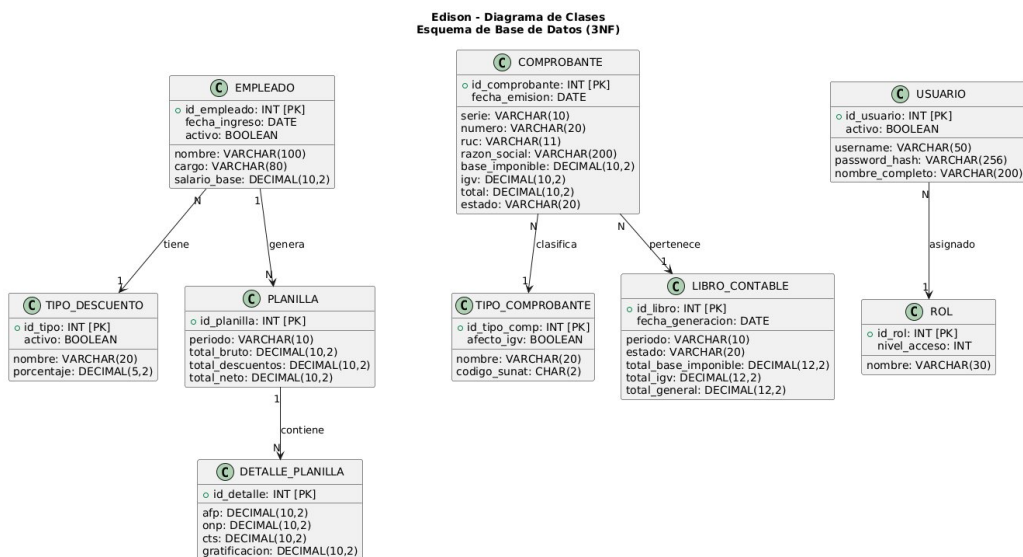
Actores:

- Arquitecto Base de Datos: Actor principal que diseña, modela y optimiza la base de datos.
- DBA Operacional: Actor secundario que realiza tareas de administración diaria (backups, monitoreo).
- PostgreSQL Servidor: Actor secundario que representa al motor de base de datos ejecutando las operaciones.

Casos de Uso:

1. Diseñar esquema normalizado 3NF: El Arquitecto diseña el esquema lógico de la base de datos aplicando la tercera forma normal, eliminando dependencias transitivas y parciales.
2. Crear stored procedures: El Arquitecto implementa procedimientos almacenados en PL/pgSQL para el cálculo de planillas (sp_calcular_planilla) y la generación de libros contables (sp_generar_libro).
3. Definir índices y constraints: El Arquitecto crea índices en columnas de búsqueda frecuente y restricciones CHECK, UNIQUE y NOT NULL para garantizar la integridad de los datos.
4. Gestionar backups y restauraciones: El DBA Operacional configura y ejecuta planes de respaldo periódicos (full semanal, incremental diario) y procedimientos de restauración documentados.
5. Monitorear rendimiento de consultas: El DBA Operacional monitorea las consultas lentas mediante pg_stat_statements, identifica cuellos de botella y optimiza los planes de ejecución.
6. Ejecutar migraciones EF Core: El Arquitecto genera y ejecuta las migraciones de Entity Framework Core para mantener sincronizado el esquema de base de datos con el modelo de clases de la aplicación.
7. Optimizar planes de ejecución: El Arquitecto analiza los planes de ejecución de las consultas críticas y ajusta índices, estadísticas y consultas para mejorar el rendimiento.

Diagrama de Clases - Esquema de Base de Datos (3NF)



Entidades del esquema de base de datos normalizado hasta Tercera Forma Normal:

Entidad (Tabla)	Atributos (Columnas)	Claves
EMPLEADO	PK id_empleado: INT nombre: VARCHAR(100) NOT NULL cargo: VARCHAR(80) NOT NULL salario_base: DECIMAL(10,2) NOT NULL FK id_tipo_descuento: INT NOT NULL fecha_ingreso: DATE NOT NULL activo: BOOLEAN DEFAULT true fecha_creacion: TIMESTAMP DEFAULT now()	PK: id_empleado FK: id_tipo_descuento → TIPO_DESCUENTO
PLANILLA	PK id_planilla: INT FK id_empleado: INT NOT NULL periodo: VARCHAR(10) NOT NULL total_bruto:	PK: id_planilla FK: id_empleado → EMPLEADO

	DECIMAL(10,2) NOT NULL total_descuentos: DECIMAL(10,2) NOT NULL total_neto: DECIMAL(10,2) NOT NULL fecha_generacion: TIMESTAMP DEFAULT now()	
DETALLE_PLANILLA	PK id_detalle: INT FK id_planilla: INT NOT NULL afp: DECIMAL(10,2) DEFAULT 0 onp: DECIMAL(10,2) DEFAULT 0 cts: DECIMAL(10,2) DEFAULT 0 gratificacion: DECIMAL(10,2) DEFAULT 0 descuentos_adicionales: DECIMAL(10,2) DEFAULT 0	PK: id_detalle FK: id_planilla → PLANILLA
TIPO_DESCUENTO	PK id_tipo: INT nombre: VARCHAR(20) NOT NULL UNIQUE porcentaje: DECIMAL(5,2) NOT NULL descripcion: VARCHAR(200) activo: BOOLEAN DEFAULT true	PK: id_tipo
COMPROBANTE	PK id_comprobante: INT FK id_tipo_comp: INT NOT NULL serie: VARCHAR(10) NOT NULL numero: VARCHAR(20) NOT NULL fecha_emision: DATE	PK: id_comprobante FK: id_tipo_comp → TIPO_COMPROBANTE

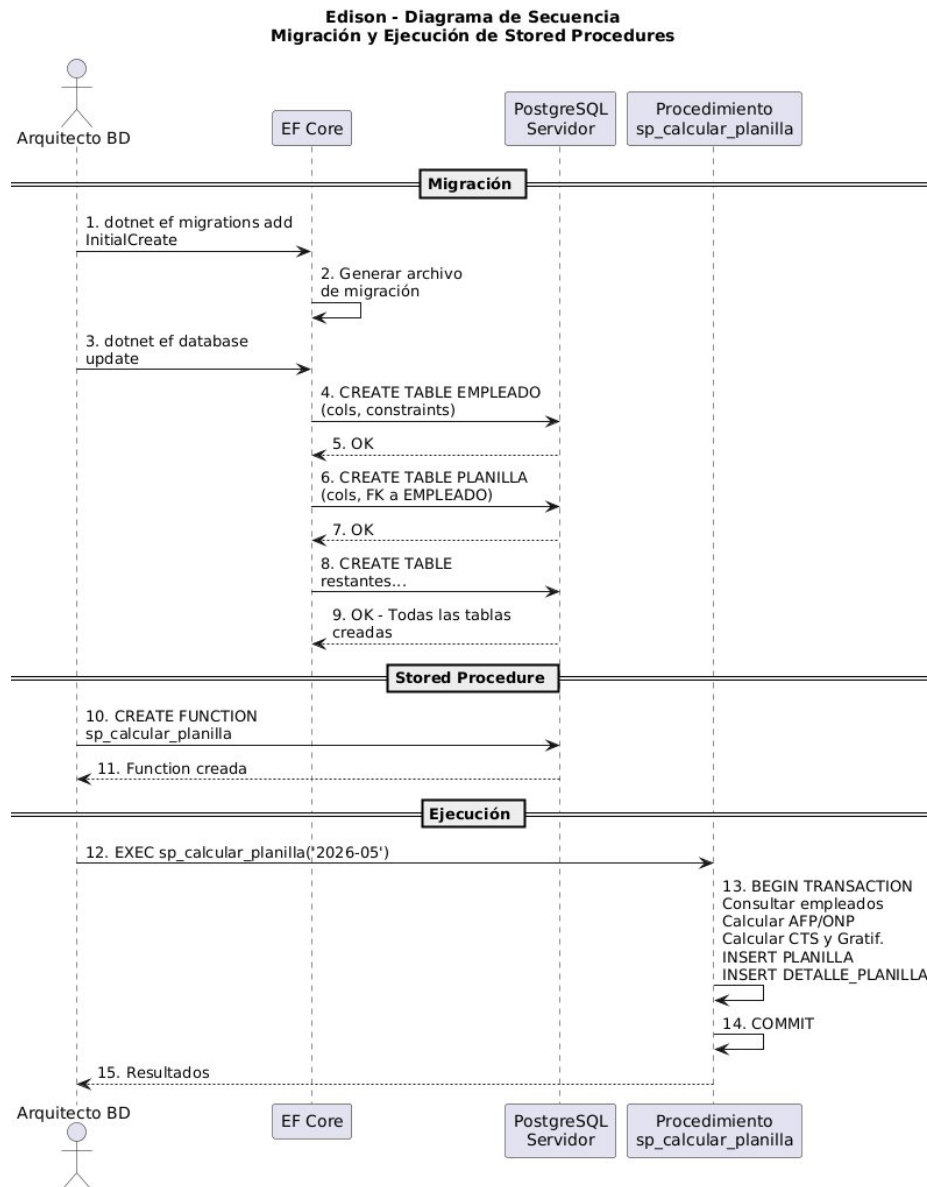
	NOT NULL ruc: VARCHAR(11) NOT NULL razon_social: VARCHAR(200) base_imponible: DECIMAL(10,2) NOT NULL igv: DECIMAL(10,2) NOT NULL total: DECIMAL(10,2) NOT NULL tipo_operacion: VARCHAR(20) estado: VARCHAR(20) DEFAULT 'Registrado'	
LIBRO_CONTABLE	PK id_libro: INT FK id_tipo_libro: INT NOT NULL periodo: VARCHAR(10) NOT NULL fecha_generacion: DATE NOT NULL estado: VARCHAR(20) DEFAULT 'Generado' total_base_imponible: DECIMAL(12,2) total_igv: DECIMAL(12,2) total_general: DECIMAL(12,2)	PK: id_libro FK: id_tipo_libro → TIPO_LIBRO
USUARIO	PK id_usuario: INT username: VARCHAR(50) NOT NULL UNIQUE password_hash: VARCHAR(256) NOT NULL nombre_completo: VARCHAR(200) NOT NULL FK id_rol: INT NOT NULL ultimo_acceso: TIMESTAMP	PK: id_usuario FK: id_rol → ROL

	activo: BOOLEAN DEFAULT true	
ROL	PK id_rol: INT nombre: VARCHAR(30) NOT NULL UNIQUE descripcion: VARCHAR(200) nivel_acceso: INT DEFAULT 1	PK: id_rol

Relaciones entre entidades:

- EMPLEADO (N) → (1) TIPO_DESCUENTO: cada empleado tiene un tipo de descuento (AFP o ONP).
- EMPLEADO (1) → (N) PLANILLA: cada empleado tiene múltiples registros de planilla mensual.
- PLANILLA (1) → (N) DETALLE_PLANILLA: cada planilla contiene un detalle por empleado (relación en 3FN).
- COMPROBANTE (N) → (1) TIPO_COMPROBANTE: cada comprobante es de un tipo específico (Factura, Boleta, NC, ND).
- LIBRO_CONTABLE (1) → (N) COMPROBANTE_LIBRO: un libro contiene múltiples comprobantes (tabla puente).
- USUARIO (N) → (1) ROL: cada usuario tiene un rol asignado.

Diagrama de Secuencia - Migración y Ejecución de Stored Procedures

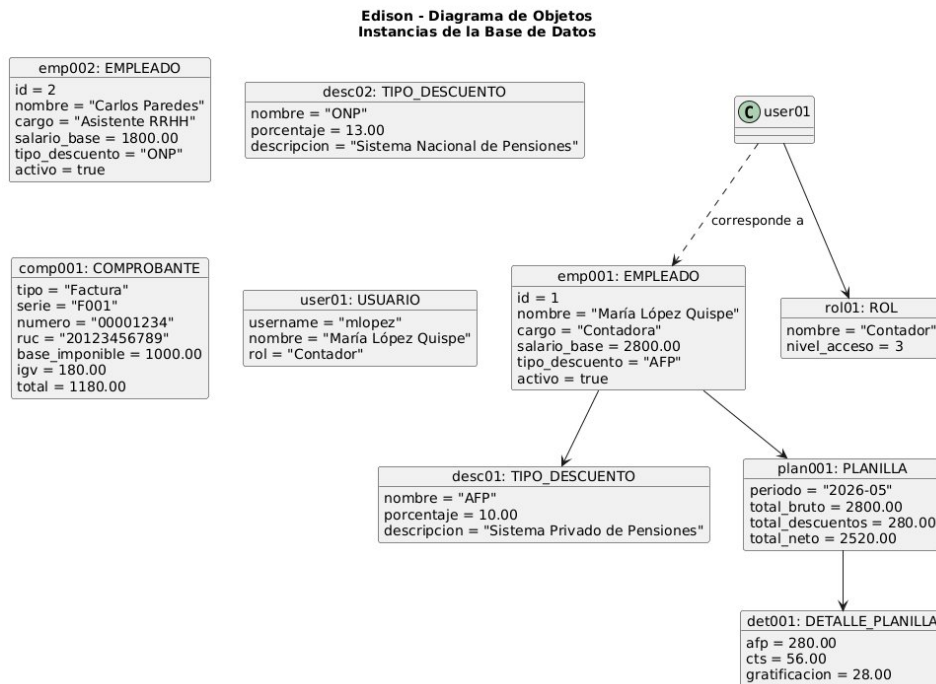


Escenario: El Arquitecto de BD ejecuta una migración de Entity Framework Core para crear las tablas iniciales y posteriormente se ejecuta el procedimiento almacenado de cálculo de planilla.

Secuencia de mensajes:

1. El Arquitecto ejecuta el comando `dotnet ef migrations add InitialCreate` desde la terminal.
2. EF Core genera el archivo de migración con las operaciones `CREATE TABLE` para todas las entidades del modelo.
3. El Arquitecto ejecuta `dotnet ef database update` para aplicar la migración a PostgreSQL.
4. PostgreSQL ejecuta los comandos DDL: `CREATE TABLE EMPLEADO` (con columnas, tipos, constraints).
5. PostgreSQL confirma la creación de la tabla `EMPLEADO`.
6. EF Core continúa con `CREATE TABLE PLANILLA` (con FK a `EMPLEADO`).
7. PostgreSQL confirma la creación.
8. El proceso se repite para todas las tablas del modelo (`DETALLE_PLANILLA`, `COMPROBANTE`, `USUARIO`, `ROL`, etc.).
9. Una vez creado el esquema, el Arquitecto ejecuta el script SQL para crear el procedimiento almacenado `sp_calcular_planilla`.
10. El procedimiento almacenado comienza una transacción.
11. Dentro de la transacción, selecciona los empleados activos con sus datos de salario y tipo de descuento.
12. Para cada empleado, calcula el salario bruto, aplica el porcentaje AFP (10 %) u ONP (13 %) según corresponda, calcula CTS y gratificación.
13. Inserta el registro en la tabla `PLANILLA` y los detalles en `DETALLE_PLANILLA`.
14. Confirma la transacción (`COMMIT`) y retorna los resultados.
15. PostgreSQL confirma la finalización exitosa del procedimiento.

Diagrama de Objetos - Instancias de la Base de Datos



Instancias de registros en las tablas de la base de datos durante la operación:

Objeto	Tabla	Atributos
emp001	EMPLEADO	id=1, nombre="María López Quispe", cargo="Contadora", salario_base=2800.00, id_tipo_descuento=1 (AFP), fecha_ingreso="2020-03-15", activo=true
emp002	EMPLEADO	id=2, nombre="Carlos Paredes Gutiérrez", cargo="Asistente RRHH", salario_base=1800.00, id_tipo_descuento=2 (ONP), fecha_ingreso="2022-06-01", activo=true
plan001	PLANILLA	id=1, id_empleado=1, periodo="2026-05", total_bruto=2800.00, total_descuentos=280.00, total_neto=2520.00

det001	DETALLE_PLANILLA	id=1, id_planilla=1, afp=280.00, onp=0.00, cts=56.00, gratificacion=28.00, desc_adicionales=0.00
desc01	TIPO_DESCUENTO	id=1, nombre="AFP", porcentaje=10.00, descripcion="Sistema Privado de Pensiones 10%"
desc02	TIPO_DESCUENTO	id=2, nombre="ONP", porcentaje=13.00, descripcion="Sistema Nacional de Pensiones 13%"
comp001	COMPROBANTE	id=1, id_tipo_comp=1 (Factura), serie="F001", numero="00001234", fecha="2026-05-15", ruc="20123456789", base_imponible=1000.00, igv=180.00, total=1180.00
user01	USUARIO	id=1, username="mlopez", nombre_completo="María López Quispe", id_rol=2 (Contador), activo=true
rol01	ROL	id=2, nombre="Contador", nivel_acceso=3

Relaciones entre instancias:

- emp001 → desc01: María López tiene tipo de descuento AFP (10 %).
- emp001 → plan001: María López tiene planilla registrada en Mayo 2026.
- plan001 → det001: la planilla de María tiene detalle con AFP de S/ 280.00.
- user01 → emp001: el usuario "mlopez" corresponde a la empleada María López.
- user01 → rol01: María tiene rol de Contador.

Normalización de Base de Datos (1FN → 2FN → 3FN → BCNF)

La normalización de la base de datos es el proceso de organizar las tablas y columnas para reducir la redundancia de datos y mejorar la integridad referencial. A continuación se presenta el proceso completo de normalización desde la tabla no normalizada hasta la Forma Boyce-Codd (BCNF).

Este proceso se detalla en la Sección 9. Normalización de Base de Datos.

Normalización de Base de Datos (Edison David Supo Cruz)

La normalización es un proceso fundamental en el diseño de bases de datos relacionales que consiste en organizar las tablas y sus columnas para eliminar redundancias, evitar anomalías de actualización y garantizar la integridad de los datos. A continuación se presenta el proceso completo aplicado al Módulo de Gestión Financiera y Contable, siguiendo las formas normales desde la tabla no normalizada hasta la Forma Boyce-Codd (BCNF).

Tabla sin normalizar (No Normalizada)

En esta etapa inicial, todos los datos del sistema se almacenan en una única tabla que contiene campos repetidos y atributos multivaluados. Esta estructura presenta graves problemas de redundancia y anomalías de actualización.

Tabla: PLANILLA_GENERAL (No Normalizada)

id_planilla	periodo	id_empleado	nombre_empleado	cargo	salario_base	tipo_descuentos	descuentos	id_comprobante	tipo_comprobante	ruc	moneto_comprobante	igv_comprobante	id_usuario	usuario	rol_usuario
1	2026-05	1	María López	Contadora	2800.00	AFP	AFP=280.00, CTS=56.00	1	Factura	20123456789	1000.00	180.00	1	mlopez	Contador
2	2026-05	2	Carlos Paredes	Asistente RRHH	1800.00	ONP	ONP=234.00	2	Boleta	10234567890	500.00	0.00	2	cparedes	Admin RRHH
1	2026-05	1	María López	Contadora	2800.00	AFP	AFP=280.00, CTS=56.00	3	Factura	20123456789	2000.00	360.00	1	mlopez	Contador

Problemas identificados:

1. Redundancia: Los datos del empleado (nombre, cargo, salario_base) se repiten cada vez que el empleado aparece en un comprobante diferente.

2. Grupos repetidos: La columna descuentos contiene múltiples valores (AFP, CTS) en un solo campo, violando la atomicidad.
3. Dependencias funcionales no identificadas: No hay una estructura clara que separe entidades independientes.
4. Anomalías de actualización: Si el salario de María López cambia, debe actualizarse en todas las filas donde aparece.
5. Anomalías de eliminación: Si se elimina el único comprobante de Carlos Paredes, se perderían todos sus datos personales.

Primera Forma Normal (1FN)

Reglas de la 1FN:

- Todos los atributos deben ser atómicos (un solo valor por campo).
- No debe haber grupos repetidos.
- Cada fila debe ser única (definir una clave primaria).

Aplicación:

1. Descomponer el campo multivaluado descuentos: Separar en columnas atómicas: afp, onp, cts, gratificacion.
2. Eliminar grupos repetidos: Separar los datos de empleados, comprobantes y usuarios en grupos distintos.
3. Definir clave primaria compuesta: (id_empleado, periodo, id_comprobante) para identificar cada fila de forma única.

Tabla resultante (parcial) – transición a 1FN:

id_empleado	periodo	nombre_empleado	cargo	salario_base	tipo_desc	afp	onp	cts	id_comprobante	tipo_comp	ruc	base_imponible	igv	id_usuario	username	rol_usuario
1	2026-05	María López	Contadora	2800.00	AFP	280.00	0.00	56.00	1	Factura	20123456789	1000.00	180.00	1	mlopez	Contador
1	2026-05	María López	Contadora	2800.00	AFP	280.00	0.00	56.00	3	Factura	20123456789	2000.00	360.00	1	mlopez	Contador
2	2026-05	Carlos Paredes	Asistente RRHH	1800.00	ONP	0.00	234.00	36.00	2	Boleta	10234567890	500.00	0.00	2	cparedes	Adm RRHH

Clave primaria compuesta: (id_empleado, periodo, id_comprobante)

Problemas remanentes (dependencias parciales):

- nombre_empleado, cargo, salario_base, tipo_desc dependen solo de id_empleado, no de toda la clave primaria.
- tipo_comp, ruc, base_imponible, igv dependen solo de id_comprobante, no de toda la clave primaria.
- username, rol_usuario dependen solo de id_usuario, no de toda la clave primaria.

Segunda Forma Normal (2FN)

Reglas de la 2FN:

- Debe estar en 1FN.
- Todos los atributos no clave deben depender funcionalmente de toda la clave primaria (eliminar dependencias parciales).

Aplicación:

Se eliminan las dependencias parciales creando tablas separadas para cada entidad que depende solo de parte de la clave primaria.

Tabla EMPLEADO:

id_empleado (PK)	nombre_empleado	cargo	salario_base	tipo_descuento
1	María López	Contadora	2800.00	AFP
2	Carlos Paredes	Asistente RRHH	1800.00	ONP

Tabla PLANILLA:

id_planilla (PK)	id_empleado (FK)	periodo	afp	onp	cts	gratificación
1	1	2026-05	280.00	0.00	56.00	28.00
2	2	2026-05	0.00	234.00	36.00	18.00

Tabla COMPROBANTE:

id_comprobante (PK)	tipo_comprobante	ruc	base_imponible	igv
1	Factura	20123456789	1000.00	180.00
2	Boleta	10234567890	500.00	0.00
3	Factura	20123456789	2000.00	360.00

Tabla USUARIO:

id_usuario (PK)	username	rol_usuario
1	mlopez	Contador
2	cparedes	Admin RRHH

Problemas remanentes (dependencias transitivas):

- En la tabla EMPLEADO: tipo_desc determina el porcentaje de descuento, pero el porcentaje no está en la tabla. Si se agregara un campo porcentaje_desc, dependería de tipo_desc, no de id_empleado (dependencia transitiva).
- En la tabla USUARIO: rol_usuario determina los permisos y nivel de acceso, pero estos no están en la tabla. Si se agregaran, dependerían de rol_usuario, no de id_usuario (dependencia transitiva).

Tercera Forma Normal (3FN)

Reglas de la 3FN:

- Debe estar en 2FN.
- Todos los atributos no clave deben depender directamente de la clave primaria (eliminar dependencias transitivas).

Aplicación:

Se eliminan las dependencias transitivas creando tablas separadas para los atributos que dependen de otros atributos no clave.

Tabla TIPO_DESCUENTO (nueva, elimina dependencia transitiva en EMPLEADO):

id_tipo (PK)	nombre	porcentaje	descripcion
1	AFP	10.00	Sistema Privado de Pensiones
2	ONP	13.00	Sistema Nacional de Pensiones

Tabla EMPLEADO (actualizada con FK a TIPO_DESCUENTO):

id_empleado (PK)	nombre_empleado	cargo	salario_base	id_tipo_descuento (FK)	fecha_ingreso
1	María López	Contadora	2800.00	1	2020-03-15
2	Carlos Paredes	Asistente RRHH	1800.00	2	2022-06-01

Tabla ROL (nueva, elimina dependencia transitiva en USUARIO):

id_rol (PK)	nombre	descripcion	nivel_acceso
1	Administrador RRHH	Gestión de empleados y planillas	2
2	Contador	Gestión de comprobantes y libros SUNAT	3
3	Gerente Financiero	Visualización de reportes financieros	1
4	Administrador Sistema	Gestión total del sistema	5

Tabla USUARIO (actualizada con FK a ROL):

id_usuario (PK)	username	password_hash	nombre_completo	id_rol (FK)	activo
1	mlopez	\$2a\$11\$...	María López Quispe	2	true
2	cparedes	\$2a\$11\$...	Carlos Paredes Gutiérrez	1	true

Tabla TIPO_COMPROBANTE (nueva, elimina dependencia transitiva en COMPROBANTE):

id_tipo_comp (PK)	nombre	codigo_sunat	afecto_igv
1	Factura	01	true
2	Boleta de Venta	03	true
3	Nota de Crédito	07	true
4	Nota de Débito	08	true
5	Guía de Remisión	09	false

Tabla COMPROBANTE (actualizada con FK a TIPO_COMPROBANTE y TIPO_LIBRO):

id_comprobante (PK)	id_tipo_comp (FK)	serie	numero	fecha_emision	ruc	razon_social	base_imponible	igv	total	tipo_operacion	estado
1	1	F00	00001234	2026-05-15	20123456789	Empresa ABC	1000.00	180.00	1180.00	Gravada	Registrado

		1				S.A.C					
2	2	B 0 0 1	000 056 78	2026- 05-16	1023 4567 890	Juan Pérez	500.00	0. 00	50 0.0 0	Exone rada	Regi stra do

Tabla LIBRO_CONTABLE:

id_libro (PK)	tipo_libro	periodo	fecha_generacion	estado
1	Compras	2026-05	2026-06-05	Generado
2	Ventas	2026-05	2026-06-05	Generado

Tabla DETALLE_PLANILLA (nueva, descompone PLANILLA para 3FN):

id_detalle (PK)	id_planilla (FK)	afp	onp	cts	gratificacion	descuentos_adicionales
1	1	280.00	0.00	56.00	28.00	0.00
2	2	0.00	234.00	36.00	18.00	0.00

Esquema Final BCNF (Boyce-Codd Normal Form)

Reglas de la BCNF:

- Debe estar en 3FN.
- Para toda dependencia funcional no trivial $X \rightarrow Y$, X debe ser una superclave de la tabla.

Verificación del esquema 3FN y ajustes para BCNF:

Todas las tablas del esquema 3FN cumplen con BCNF porque en cada tabla, los únicos determinantes de atributos no clave son las claves primarias (superclaves). No existen dependencias funcionales donde un atributo no clave determine otro atributo no clave.

Esquema final completo de la base de datos en BCNF:

Claves Foráneas y Restricciones de Integridad:

Tabla	Clave Foránea	Tabla Referenciada	Acción ON DELETE	Acción ON UPDATE
-------	---------------	--------------------	------------------	------------------

EMPLEADO	id_tipo_descuento	TIPO_DESCUENTO(id_tipo)	RESTR ICT	CASCADE
PLANILLA	id_empleado	EMPLEADO(id_empleado)	CASCADE	CASCADE
DETALLE_PLANILLA	id_planilla	PLANILLA(id_planilla)	CASCADE	CASCADE
COMPROBANTE	id_tipo_comprobante	TIPO_COMPROBANTE(id_tipo)	RESTR ICT	CASCADE
COMPROBANTE	id_libro	LIBRO_CONTABLE(id_libro)	RESTR ICT	CASCADE
LIBRO_CONTABLE	id_tipo_libro	TIPO_LIBRO(id_tipo)	RESTR ICT	CASCADE
USUARIO	id_rol	ROL(id_rol)	RESTR ICT	CASCADE

Restricciones adicionales (CHECK y UNIQUE):

Procedimiento Almacenado sp_calcular_planilla (PL/pgSQL):

Ventajas del esquema normalizado BCNF:

1. Eliminación de redundancia: Los datos de cada entidad se almacenan una sola vez.
2. Prevención de anomalías: Las actualizaciones, inserciones y eliminaciones no causan inconsistencias.
3. Integridad referencial: Las claves foráneas garantizan que las relaciones entre tablas sean válidas.
4. Eficiencia de consultas: Los índices definidos sobre claves primarias y foráneas optimizan las operaciones JOIN.
5. Escalabilidad: El esquema puede extenderse agregando nuevas tablas sin modificar las existentes.
6. Cumplimiento de BCNF: Toda dependencia funcional no trivial tiene una superclave como determinante.

Referencias

SUNAT. (2026). Normativa tributaria peruana – Libros Contables Electrónicos. Lima, Perú: Superintendencia Nacional de Aduanas y de Administración Tributaria.

Schwaber, K., & Sutherland, J. (2020). La Guía Scrum: Las reglas del juego. Scrum.org. Recuperado de <https://scrumguides.org/>

Microsoft. (2024). C# programming guide (.NET documentation). Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/dotnet/csharp/>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Recuperado de <https://www.postgresql.org/docs/16/>

Pressman, R. S., & Maxim, B. R. (2015). Ingeniería del software: Un enfoque práctico (8.^a ed.). McGraw-Hill.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.

Microsoft. (2024). Entity Framework Core documentation. Microsoft Learn. Recuperado de <https://learn.microsoft.com/en-us/ef/core/>

Entregables del Sprint 0

Anexo 1: Hoja de Identificación del Proyecto

Información General del Proyecto

Campo	Valor
Nombre del Proyecto	Sistema ERP – Módulo de Gestión Financiera y Contable
Código del Proyecto	ERP-GFC-001-2026
Líder de Proyecto	Alvis Danival Quispe Huayta
Fecha de Creación	23 de mayo de 2026
Versión	1.0.0.0
Curso	Taller de Procesos ERP / ERP Process Workshop
Docente	M.Sc. Juan Carlos Pinto Larico
Institución	Universidad Nacional de Juliaca (UNAJ)
Facultad	Ciencias de la Ingeniería
Escuela Profesional	Ingeniería de Software y Sistemas

Matriz de Requisitos Funcionales

ID	Nombre del Requisito	Descripción Breve	Prioridad	Sprint
RF-001	Gestión de Empleados	Registrar, editar y consultar datos del personal	Alta	Sprint 1
RF-002	Cálculo de Planillas	Cálculo automático de sueldos, AFP, ONP, CTS	Alta	Sprint 1
RF-003	Reporte de Pago	Exportar boletas de pago en PDF	Alta	Sprint 1
RF-004	Registro Contable	Registrar comprobantes de compra y venta	Alta	Sprint 2
RF-005	Libros SUNAT	Generar Libro de Compras y Libro de Ventas	Alta	Sprint 2

RF-006	Normativa Tributaria	Aplicar tasas IGV y renta vigentes según SUNAT	Alta	Sprint 2
RF-007	Reportes Financieros	Generar balance general y estado de resultados	Media	Sprint 3
RF-008	Autenticación	Login con credenciales y control de acceso por roles	Alta	Sprint 1

Matriz de Requisitos No Funcionales

ID	Categoría	Criterio Medible	Descripción
RNF-001	Rendimiento	Cálculo planilla ≤ 30 s / 100 empleados	El sistema debe calcular la planilla de 100 empleados en máximo 30 segundos
RNF-002	Seguridad	Cifrado AES-256, autenticación JWT	El sistema debe cifrar credenciales con AES-256 y usar JWT para autenticación
RNF-003	Usabilidad	Tareas frecuentes en ≤ 3 clics	La interfaz debe permitir tareas frecuentes en máximo 3 interacciones
RNF-004	Compatibilidad	Windows 10+, .NET 8	El sistema debe operar en Windows 10 y superiores con .NET 8
RNF-005	Mantenibilidad	Arquitectura modular independiente	Actualizar un módulo no debe afectar los demás

Cronograma Tentativo

Sprint	Período	Módulo	Entregable
Sprint 0	Mayo 2026	Planificación y Análisis	Documentación inicial, requisitos, diagramas UML
Sprint 1	Junio 2026	Planillas	CRUD empleados, cálculo automático, boletas PDF
Sprint 2	Julio 2026	Contable SUNAT	Comprobantes, Libro Compras, Libro Ventas
Sprint 3	Agosto 2026	Reportes	Dashboard, balance general, estado de resultados

Anexo 2: Hoja de Diseño Técnico

Proyecto: Sistema ERP – Módulo de Gestión Financiera y Contable

Tecnologías: C# .NET Core 8, WinForms, PostgreSQL 16, Entity Framework Core 8, PL/pgSQL

Metodología: Ágil – Scrum (Sprints de 4 semanas)

Versión: 1.0.0

Autor: Equipo de desarrollo – UNAJ

Fecha: Mayo 2026

Introducción

Este documento describe el diseño técnico detallado del Módulo de Gestión Financiera y Contable del sistema ERP, incluyendo la arquitectura de software, los componentes principales, los patrones de diseño aplicados, los diagramas de interacción y las consideraciones técnicas para la implementación. El objetivo es garantizar una implementación modular, escalable, segura y mantenible que cumpla con los requisitos del negocio y la normativa SUNAT.

Arquitectura del Sistema (Tres Capas + Patrón MVC)

Capa de Presentación (UI):

- Tecnología: Windows Forms (WinForms) con .NET Core 8.

- Patrón: Model-View-ViewModel (MVVM) para separar la lógica de presentación de la interfaz visual.
- Componentes:
 - MainWindow: Ventana principal con menú lateral de navegación.
 - PlanillaView: Vista de gestión de empleados y cálculo de planillas.
 - ContableView: Vista de registro de comprobantes y generación de libros SUNAT.
 - ReportesView: Vista de dashboards y reportes financieros.
 - AdminView: Vista de administración de usuarios y roles.
- Funcionalidades: formularios de entrada con validación en tiempo real, tablas con ordenamiento y filtros, gráficos dinámicos, notificaciones al usuario.

Capa de Lógica de Negocio (API Backend):

- Tecnología: ASP.NET Core 8 Web API.
- Patrones: Repository Pattern, Dependency Injection, Service Layer.
- Módulos:
 - AuthModule: Autenticación JWT, gestión de sesiones, control de roles (RBAC).
 - PlanillaModule: Cálculo de planillas, registro de empleados, generación de boletas PDF.
 - ContableModule: Registro de comprobantes, cálculo de IGV, generación de libros SUNAT.
 - ReporteModule: Generación de balance general, estado de resultados, reportes exportables.
- Formato de intercambio: JSON (API RESTful).
- Documentación: Swagger UI / OpenAPI 3.0.

Capa de Acceso a Datos:

- Tecnología: Entity Framework Core 8 con proveedor Npgsql para PostgreSQL.
- Patrones: Repository Pattern, Unit of Work.
- Componentes:
 - ErpDbContext: Contexto de base de datos que mapea las entidades del modelo.
 - Migraciones automatizadas con EF Core Code-First.

- Procedimientos almacenados PL/pgSQL para operaciones críticas (cálculo de planilla, generación de libros).
- Base de datos: PostgreSQL 16 con esquema normalizado BCNF.

Componentes Principales

Componente	Descripción	Tecnología	Patrón
Interfaz WinForms	Aplicación de escritorio con navegación por módulos	WinForms + .NET 8	MVVM
API Gateway	Punto de entrada único, enrutamiento, middleware	ASP.NET Core 8	Middleware Pipeline
AuthService	Autenticación JWT, BCrypt, gestión de roles	C# + JWT Bearer	Service Layer
PlanillaService	Lógica de cálculo de planillas y empleados	C# .NET 8	Service Layer
ContableService	Lógica de contabilidad SUNAT	C# .NET 8	Service Layer
ReporteService	Generación de reportes financieros	C# + QuestPDF/EPPlus	Service Layer
ErpDbContext	Contexto de EF Core para PostgreSQL	EF Core 8 + Npgsql	Repository + UoW
PostgreSQL 16	Motor de base de datos relacional	PostgreSQL 16	-
Redis Cache	Caché de sesiones y rate limiting	StackExchange.Redis	Cache-Aside

Diagrama de Interacción (Secuencia) - Generación del Libro de Ventas Mensual

Escenario: El Contador genera el Libro de Ventas del período Mayo 2026.

1. El Contador selecciona "Generar Libro de Ventas" en la interfaz ContableView y especifica el período "2026-05".
2. ContableView envía una solicitud GET al endpoint /api/libros/ventas?periodo=2026-05 del backend, incluyendo el token JWT en el encabezado.
3. El API Gateway recibe la solicitud, el middleware de autenticación valida el token y verifica que el usuario tenga rol de Contador.
4. El gateway enruta la solicitud al ContableController, que invoca ContableService.GenerarLibroVentas(periodo).
5. ContableService consulta todos los comprobantes de venta del período con estado "Registrado" mediante ContableRepository.
6. ContableRepository ejecuta la consulta SQL: SELECT FROM COMPROBANTE WHERE id_tipo_comp IN (1,2,3,4) AND fecha_emision BETWEEN '2026-05-01' AND '2026-05-31' AND estado = 'Registrado'.
7. PostgreSQL retorna los comprobantes del período.
8. ContableService estructura los datos según el formato SUNAT de Libro de Ventas Electrónico: agrupa por tipo de comprobante, calcula totales (base imponible, IGV, total) por grupo y genera el libro.
9. ContableService guarda el libro generado en la tabla LIBRO_CONTABLE con estado "Generado".
10. ContableController retorna una respuesta HTTP 200 OK con el libro de ventas estructurado en JSON.
11. ContableView muestra el Libro de Ventas en una tabla con totales calculados al pie.
12. El Contador puede hacer clic en "Exportar PDF" para descargar el libro.

Consideraciones Técnicas

Aspecto	Especificación
Lenguaje	C# (.NET Core 8)
IDE	Visual Studio 2022 / Visual Studio Code
Base de datos	PostgreSQL 16 con conexión SSL
ORM	Entity Framework Core 8 (Code-First)
Autenticación	JWT (JSON Web Tokens) con BCrypt para hash de contraseñas
Generación PDF	QuestPDF / iTextSharp
Generación Excel	EPPlus / ClosedXML

Control de versiones	Git + GitHub (commits semánticos)
Metodología	Scrum (Sprints de 4 semanas)
Patrones de diseño	MVC, MVVM, Repository, Singleton, Factory, Dependency Injection

Criterios de Aceptación Técnicos

1. La arquitectura debe permitir agregar nuevos módulos (inventarios, CRM, RRHH) sin modificar la lógica del módulo financiero.
2. Cada operación contable y de planilla debe registrarse con trazabilidad completa (usuario, fecha, tipo de operación, resultado).
3. El Libro de Compras y Ventas debe cumplir estrictamente el formato vigente SUNAT para libros electrónicos (Resolución de Superintendencia N.º 286-2009/SUNAT).
4. El cálculo de planillas debe ser verificable con una tolerancia máxima de $\pm S/0.01$ por empleado.
5. Todos los endpoints de la API deben estar protegidos por autenticación JWT, excepto el endpoint de login.
6. La base de datos debe implementar integridad referencial mediante claves foráneas con acciones ON DELETE RESTRICT y ON UPDATE CASCADE.
7. El sistema debe manejar adecuadamente las excepciones y retornar mensajes de error descriptivos al usuario.
8. El tiempo de respuesta de la API para consultas de lectura debe ser inferior a 500 ms (percentil 95).

Anexo 3: Identificación de Requisitos Funcionales

Instrumento 1: Identificación de Requisitos Funcionales

Contexto del sistema

- Nombre del sistema: SISTEMA ERP - MÓDULO DE GESTIÓN FINANCIERA Y CONTABLE
- Tecnologías: C# (.NET Core 8), PostgreSQL 16, Entity Framework Core, WinForms, PL/pgSQL
- Breve descripción: Módulo ERP para la gestión automatizada de planillas de personal (cálculo de sueldos, AFP, ONP, CTS, gratificaciones) y contabilidad SUNAT (registro de comprobantes, generación de Libro de Compras y Libro de Ventas electrónicos, reportes financieros).

Pasos de trabajo

Paso 1: Identificar actores principales

Pregunta	Respuesta
¿Quiénes usarán el sistema?	Administrador RRHH, Contador, Gerente Financiero, Administrador del Sistema
¿Qué roles tienen?	RRHH gestiona empleados y planillas; Contador gestiona comprobantes y libros SUNAT; Gerente visualiza reportes financieros; Administrador gestiona usuarios y configuraciones

Paso 2: Definir funciones esperadas

Pregunta	Respuesta
¿Qué tareas debe permitir?	Registrar empleados, calcular planillas mensuales, registrar comprobantes de compra/venta, generar Libro de Compras y Ventas, visualizar dashboards financieros, exportar reportes a PDF/Excel
¿Qué entradas/salidas?	Entrada: datos de empleados, comprobantes, configuraciones. Salida: planillas PDF, libros contables (Compras/Ventas), balance general, estado de resultados

Paso 3: Redactar requisitos funcionales

Formato: "El sistema debe permitir [acción] para [actor] con el fin de [objetivo]"

Tabla de Registro de Requisitos Funcionales

Nº	Actor	Acción esperada	Requisito funcional redactado
1	Administrador RRHH	Registrar datos de empleados	El sistema debe permitir registrar, editar y consultar datos del personal para el Administrador

			RRHH con el fin de mantener actualizada la nómina de la organización
2	Administrador RRHH	Calcular planilla mensual	El sistema debe permitir calcular automáticamente sueldos, AFP, ONP, CTS y gratificaciones para el Administrador RRHH con el fin de generar la planilla mensual precisa
3	Administrador RRHH	Generar reporte de pago	El sistema debe permitir generar boletas de pago en formato PDF para el Administrador RRHH con el fin de documentar y distribuir los pagos del personal
4	Contador	Registrar comprobantes	El sistema debe permitir registrar comprobantes de compra y venta para el Contador con el fin de mantener el registro contable actualizado conforme a la normativa SUNAT
5	Contador	Generar libros SUNAT	El sistema debe permitir generar el Libro de Compras y el Libro de Ventas electrónicos para el Contador con el

			fin de cumplir las obligaciones tributarias ante la SUNAT
6	Gerente Financiero	Visualizar reportes	El sistema debe permitir generar y visualizar el balance general y el estado de resultados para el Gerente Financiero con el fin de apoyar la toma de decisiones estratégicas
7	Administrador Sistema	Gestionar accesos	El sistema debe permitir crear, modificar y desactivar cuentas de usuario con asignación de roles para el Administrador del Sistema con el fin de controlar el acceso a los módulos del sistema

Anexo 4: Definición de Requisitos No Funcionales

Instrumento 2: Definición de Requisitos No Funcionales

Contexto del sistema

- Nombre del sistema: SISTEMA ERP - MÓDULO DE GESTIÓN FINANCIERA Y CONTABLE
- Resolución de interfaz: 1280 × 720 píxeles (mínimo), adaptable hasta 1920 × 1080 píxeles
- Breve descripción: Sistema de escritorio en C# con WinForms para gestión de planillas y contabilidad SUNAT, con backend ASP.NET Core 8 y base de datos PostgreSQL 16

Categorías de requisitos no funcionales

Categoría	Descripción
-----------	-------------

Rendimiento	Tiempo de respuesta, capacidad de usuarios concurrentes, velocidad de procesamiento de cálculos
Seguridad	Autenticación, cifrado de datos, control de acceso basado en roles, auditoría de operaciones
Usabilidad	Facilidad de uso, accesibilidad, interfaz intuitiva, consistencia visual, tiempo de aprendizaje
Compatibilidad	Sistemas operativos soportados, versiones de .NET runtime, resoluciones de pantalla
Mantenibilidad	Modularidad del código, documentación, pruebas automatizadas, facilidad de actualización

Pasos de trabajo

1. Seleccionar categorías relevantes para el sistema financiero-contable.
2. Definir criterios medibles para cada categoría (ejemplo: "cálculo de planilla < 30 segundos para 100 empleados").
3. Redactar el requisito no funcional con el formato: "El sistema debe [característica] con un nivel de [criterio medible]".

Tabla de Registro de Requisitos No Funcionales

Nº	Categoría	Criterio Medible	Requisito No Funcional Redactado
1	Rendimiento	Tiempo de cálculo de planilla ≤ 30 segundos para 100 empleados	El sistema debe calcular la planilla de hasta 100 empleados con un nivel de rendimiento que garantice un tiempo de respuesta máximo de 30 segundos desde la solicitud hasta la visualización de resultados
2	Seguridad	Cifrado AES-256	El sistema debe

		en reposo y TLS 1.3 en tránsito, autenticación JWT con BCrypt	proteger las credenciales de usuario y los datos financieros con un nivel de seguridad que incluya cifrado AES-256 para datos almacenados, TLS 1.3 para comunicaciones de red, hash BCrypt para contraseñas y autenticación mediante tokens JWT con expiración
3	Usabilidad	Tareas frecuentes completables en máximo 3 clics, tiempo de aprendizaje menor a 15 minutos	El sistema debe ofrecer una experiencia de usuario con un nivel de usabilidad que permita completar las tareas más frecuentes (registrar empleado, calcular planilla, ver reportes) en no más de 3 interacciones desde la pantalla principal
4	Compatibilidad	Windows 10 y 11, .NET 8 Runtime, resolución 1280×720 a 1920×1080	El sistema debe operar correctamente con un nivel de compatibilidad que incluya los sistemas operativos Windows 10 (versión 22H2) y

			Windows 11, requiriendo .NET 8 Runtime y adaptándose a resoluciones de pantalla desde 1280×720 hasta 1920×1080
5	Mantenibilidad	Arquitectura modular con módulos independientes, cobertura de pruebas > 70 %	El sistema debe implementar una arquitectura de software con un nivel de mantenibilidad que permita actualizar, corregir o reemplazar cualquiera de los módulos (planillas, contable, reportes, autenticación) sin afectar el funcionamiento de los demás módulos

Anexo 5: Wireframes y Pantallas del Sistema

Este anexo describe en detalle las pantallas y wireframes del Sistema ERP - Módulo de Gestión Financiera y Contable, incluyendo los elementos de la interfaz, los flujos de navegación y las funcionalidades disponibles en cada pantalla. Los wireframes presentados corresponden a un nivel de fidelidad media-alta, diseñados para guiar la implementación en WinForms durante el Sprint 1.

Pantalla de Login / Inicio de Sesión

Descripción: Pantalla inicial del sistema donde los usuarios se autentican para acceder a los módulos según su rol.

Elementos de la interfaz:

- Logo institucional: Logotipo de la UNAJ y nombre del sistema "ERP - Gestión Financiera y Contable" en la parte superior.

- Campo de usuario: TextBox con etiqueta "Usuario" y placeholder "Ingresa su nombre de usuario".
- Campo de contraseña: PasswordBox con etiqueta "Contraseña" y placeholder "Ingresa su contraseña".
- Botón "Iniciar Sesión": Botón principal con color institucional azul (#1F5C99) que ejecuta la autenticación.
- Enlace "¿Olvidó su contraseña?": Enlace que permite al usuario solicitar un reestablecimiento de contraseña (funcionalidad futura).
- Mensajes de error: Área de texto rojo que muestra mensajes de error ("Usuario o contraseña incorrectos", "Cuenta desactivada", "Campos obligatorios vacíos").

Flujo de interacción:

1. El usuario ingresa su nombre de usuario en el campo correspondiente.
2. El usuario ingresa su contraseña en el campo correspondiente.
3. El usuario hace clic en "Iniciar Sesión".
4. El sistema valida que ambos campos no estén vacíos (validación del lado del cliente).
5. El sistema envía las credenciales al backend mediante POST /api/auth/login.
6. Si la autenticación es exitosa, el sistema redirige al Dashboard principal.
7. Si la autenticación falla, el sistema muestra un mensaje de error específico.

Validaciones:

- Campos obligatorios: usuario y contraseña no pueden estar vacíos.
- Formato de usuario: al menos 3 caracteres alfanuméricos.
- Intentos fallidos: después de 3 intentos fallidos, la cuenta se bloquea por 15 minutos.

Pantalla Dashboard Principal

Descripción: Pantalla principal del sistema que se muestra después de la autenticación exitosa. Proporciona acceso a los módulos disponibles según el rol del usuario y muestra indicadores clave del sistema.

Elementos de la interfaz:

- Barra superior: Contiene el logo del sistema, el nombre del usuario autenticado, el rol y un botón "Cerrar Sesión".
- Menú lateral: Panel de navegación con iconos y nombres de los módulos disponibles:

- "Planillas" (icono de calculadora/nómina) – visible para Admin RRHH y Administrador.
- "Contabilidad SUNAT" (icono de libro contable) – visible para Contador y Administrador.
- "Reportes Financieros" (icono de gráfico) – visible para Gerente Financiero y Administrador.
- "Administración" (icono de engranaje/usuario) – visible solo para Administrador del Sistema.
- Panel de indicadores (KPIs): Cuatro tarjetas con indicadores numéricos:
 - "Total Planilla Mes": monto total de la planilla del mes actual.
 - "Empleados Activos": número total de empleados activos.
 - "Comprobantes Mes": cantidad de comprobantes registrados en el mes.
 - "IGV por Pagar": monto total de IGV por pagar en el período.
- Área de bienvenida: Mensaje de bienvenida personalizado: "Bienvenido, [Nombre del Usuario]. Seleccione un módulo para comenzar."

Flujo de interacción:

1. El usuario visualiza los indicadores clave del sistema en el panel superior.
2. El usuario selecciona un módulo del menú lateral para acceder a sus funcionalidades.
3. El sistema carga la vista correspondiente al módulo seleccionado en el área principal.
4. El usuario puede cerrar sesión haciendo clic en "Cerrar Sesión" en la barra superior.

Pantalla Módulo Planillas - Lista de Empleados

Descripción: Pantalla que muestra la lista de empleados registrados con opciones de búsqueda, filtrado y acciones CRUD.

Elementos de la interfaz:

- Barra de herramientas: Contiene:
 - Botón "Nuevo Empleado" (icono +): abre el formulario de registro.
 - Botón "Editar" (icono lápiz): abre el formulario de edición del empleado seleccionado.

- Botón "Desactivar" (icono X): desactiva al empleado seleccionado (confirmación requerida).
- Botón "Actualizar" (icono de flecha circular): recarga la lista de empleados desde el backend.
- Campo de búsqueda: TextBox con placeholder "Buscar por nombre, cargo o DNI" que filtra la lista en tiempo real.
- Filtros avanzados: Selectores desplegables para filtrar por cargo, tipo de descuento (AFP/ONP) y estado (activo/inactivo).
- Tabla de empleados: DataGridView con las siguientes columnas:
 - Nº (número correlativo)
 - DNI (documento de identidad)
 - Nombres y Apellidos
 - Cargo
 - Salario Base (formato moneda S/ X,XXX.XX)
 - Tipo Descuento (AFP / ONP)
 - Fecha Ingreso
 - Estado (Activo / Inactivo)
- Paginación: Controles de paginación (anterior/siguiente) y selector de filas por página (10, 25, 50).
- Pie de tabla: Muestra el total de empleados activos y el total de salario base mensual.

Flujo de interacción:

1. El Administrador RRHH accede al módulo Planillas y selecciona "Empleados".
2. El sistema carga y muestra la lista de empleados activos.
3. El administrador puede buscar empleados por nombre, cargo o DNI usando el campo de búsqueda.
4. El administrador puede hacer clic en "Nuevo Empleado" para abrir el formulario de registro.
5. El administrador puede seleccionar un empleado y hacer clic en "Editar" para modificar sus datos.
6. El administrador puede seleccionar un empleado y hacer clic en "Desactivar" para darlo de baja (ocultación lógica).

Pantalla Formulario de Empleado

Descripción: Formulario para registrar o editar los datos de un empleado.

Elementos de la interfaz:

- Título: "Nuevo Empleado" o "Editar Empleado: [Nombre]" según la acción.
- Campos del formulario (organizados en dos columnas):
 - Columna izquierda:
 - DNI: TextBox con máscara de 8 dígitos (formato: #####).
 - Nombres: TextBox con validación de solo letras y espacios.
 - Apellidos: TextBox con validación de solo letras y espacios.
 - Fecha de Nacimiento: DatePicker.
 - Fecha de Ingreso: DatePicker.
 - Columna derecha:
 - Cargo: ComboBox desplegable con cargos predefinidos (Contador, Asistente RRHH, Analista Financiero, etc.) y opción "Otro".
 - Salario Base: TextBox con formato moneda y validación de valor positivo.
 - Tipo Descuento: RadioButton (AFP / ONP).
 - Banco: ComboBox con lista de bancos (BCP, BBVA, Interbank, Scotiabank, etc.).
 - Número de Cuenta: TextBox con validación de 14 a 20 dígitos.
- Botones de acción:
 - "Guardar": Valida los campos y envía los datos al backend.
 - "Cancelar": Retorna a la lista de empleados sin guardar cambios.

Validaciones:

- DNI: obligatorio, 8 dígitos numéricos, único en el sistema.
- Nombres y Apellidos: obligatorios, solo letras y espacios, mínimo 2 caracteres.
- Fecha de Nacimiento: obligatorio, fecha válida, edad mínima 18 años.
- Fecha de Ingreso: obligatorio, no puede ser futura.
- Salario Base: obligatorio, valor mayor a S/ 0.00.
- Tipo Descuento: obligatorio (debe seleccionar AFP u ONP).
- Número de Cuenta: obligatorio, formato válido según banco seleccionado.

Pantalla Cálculo de Planilla

Descripción: Pantalla donde el Administrador RRHH ejecuta el cálculo de la planilla mensual y visualiza los resultados.

Elementos de la interfaz:

- Selector de período: Dos ComboBox para seleccionar mes (Enero-Diciembre) y año (2026).
- Botón "Calcular Planilla": Botón principal que ejecuta el cálculo.
- Indicador de progreso: Barra de progreso que muestra el avance del cálculo (visible durante el procesamiento).
- Tabla de resultados: DataGrid con las siguientes columnas por empleado:
 - N°, Nombre, Cargo, Salario Bruto (S/), AFP (S/), ONP (S/), CTS (S/), Gratificación (S/), Total Descuentos (S/), Neto a Pagar (S/)
- Panel de totales (al pie de la tabla):
 - Total Bruto: suma de todos los salarios brutos.
 - Total AFP: suma de todos los descuentos AFP.
 - Total ONP: suma de todos los descuentos ONP.
 - Total CTS: suma de todas las CTS.
 - Total Gratificaciones: suma de todas las gratificaciones.
 - Total Descuentos: suma de todos los descuentos.
 - Total Neto: suma de todos los salarios netos.
- Botones de exportación:
 - "Exportar Boletas PDF": genera un archivo ZIP con las boletas de pago individuales en PDF.
 - "Exportar Planilla Excel": descarga la planilla completa en formato Excel.

Flujo de interacción:

1. El Administrador RRHH selecciona el período (mes y año) y hace clic en "Calcular Planilla".
2. El sistema muestra una barra de progreso mientras el backend procesa el cálculo.
3. Al completarse, el sistema muestra la tabla con los resultados de todos los empleados activos.
4. El administrador verifica los totales y puede exportar las boletas individuales en PDF.

5. El administrador puede descargar la planilla completa en Excel para análisis adicional.

Pantalla Módulo Contable - Lista de Comprobantes

Descripción: Pantalla que muestra los comprobantes de compra y venta registrados, con opciones de filtrado y acciones CRUD.

Elementos de la interfaz:

- Barra de herramientas: Botones "Nuevo Comprobante", "Editar", "Eliminar", "Actualizar".
- Filtros: Selector de tipo de comprobante (Factura, Boleta, NC, ND, Todos), rango de fechas (desde/hasta), y campo de búsqueda por RUC o razón social.
- Tabla de comprobantes: DataGrid con columnas:
 - Tipo, Serie, Número, Fecha, RUC, Razón Social, Base Imponible (S/), IGV (S/), Total (S/), Estado (Registrado/Validado/Anulado)
- Pie de tabla: Totales de base imponible, IGV y total general del período filtrado.

Pantalla Generación de Libros SUNAT

Descripción: Pantalla para generar el Libro de Compras y Libro de Ventas electrónicos.

Elementos de la interfaz:

- Selector de período: Mes y año para la generación del libro.
- Botones de acción:
 - "Generar Libro de Compras"
 - "Generar Libro de Ventas"
- Vista previa del libro: Tabla con el formato SUNAT estándar que incluye:
 - Período, RUC, Tipo Comprobante, Serie, Número, Fecha, Base Imponible Gravada, Base Imponible Exonerada, IGV, Total
- Panel de totales: Base imponible total, IGV total, Total general.
- Botones de exportación: "Exportar PDF", "Exportar Excel".

Pantalla Dashboard de Reportes Financieros

Descripción: Pantalla que presenta los indicadores financieros clave y reportes ejecutivos para el Gerente Financiero.

Elementos de la interfaz:

- Selector de período: Rango de fechas para los reportes.

- Panel de KPIs: Tarjetas con indicadores:
 - Ingresos Totales (S/)
 - Costos y Gastos (S/)
 - Utilidad Neta (S/)
 - Margen de Utilidad (%)
 - Total Planilla (S/)
 - IGV por Pagar (S/)
- Gráficos: Gráfico de barras de ingresos vs. gastos por mes, gráfico circular de distribución de costos.
- Reportes detallados: Enlaces o botones para "Ver Balance General" y "Ver Estado de Resultados".
- Botones de exportación: Exportar reportes a PDF o Excel.

Pantalla de Administración de Usuarios

Descripción: Pantalla para gestionar usuarios, roles y permisos del sistema.

Elementos de la interfaz:

- Tabla de usuarios: DataGrid con columnas: Usuario, Nombre Completo, Rol, Último Acceso, Estado (Activo/Inactivo).
- Botones: "Nuevo Usuario", "Editar", "Desactivar", "Resetear Contraseña".
- Formulario de usuario: Campos para username, contraseña, nombre completo, selección de rol (ComboBox con roles disponibles).
- Sección de Roles: Tabla de roles con nombre, descripción y nivel de acceso, con opción de agregar/editar roles.

Documento generado el 27 de mayo de 2026.

Universidad Nacional de Juliaca - Escuela Profesional de Ingeniería de Software y Sistemas

Curso: Taller de Procesos ERP - Docente: M.Sc. Juan Carlos Pinto Larico